

Das ist das fehlende Puzzleteil! Vielen Dank für die Dateien und den HTML-Auszug, das hat den Fehler sofort entlarvt.

Hier trafen gleich zwei fiese Kantenfälle deines Export-Systems (vermutlich DOORS) aufeinander:

1. **Das versteckte Verzeichnis:** Die Verzeichnisse in deinem Dokument stecken technisch gesehen in speziellen Word-Containern (sogenannten *Structured Document Tags / SDTs*). Die normale Einlese-Schleife von Python hat diese Container übersprungen, weshalb das Inhaltsverzeichnis gar nicht erst im Vergleich auftauchte.
2. **Die Überschrift "Inhalt":** Das Skript kannte bisher nur Begriffe wie "Inhaltsverzeichnis" oder "Table of Contents". Da es "Inhalt" nicht als Verzeichnis-Start erkannte, geriet die gesamte Seitenlogik aus dem Takt und versuchte, die Verzeichniseinträge wie 1. Scope 4 als echte Kapitel zu behandeln.

Ich habe das nun komplett und robust gelöst (**Core Version 3.10 / GUI Version 2.7**):

- Das Skript knackt nun die versteckten Word-Container auf und liest die Verzeichnisse restlos aus.
- "Inhalt" ist nun als Schlüsselwort hinterlegt.
- Das Inhalts- und Abbildungsverzeichnis wird ab sofort nicht mehr unsichtbar übersprungen, sondern als **zusammenhängender, vergleichbarer Block ("Inhalt") ganz oben in deinen HTML-Report** gepackt. Du siehst also künftig auch, ob sich im Verzeichnis selbst etwas verändert hat!
- Die Word-Seitenerkennung für mehrzeilige Kapitel (wie 60.2 mit den Zeilenumbrüchen Asd und saD) ist nun unzerstörbar: Sie sucht immer nur nach der längsten der ersten drei Zeilen.

Tausche die beiden Dateien einfach durch diese neuen Versionen aus, dann wird alles sauber dargestellt.

1. Die Datei docx_chapter_compare.py (Version 3.10)

```
#!/usr/bin/env python3
"""
docx_chapter_compare.py
Version 3.10 / 2026-09-21 / Grund: Bugfix für Inhalts- und
Abbildungsverzeichnisse
    in speziellen Export-Formaten (z.B. DOORS). 1) Das Skript
extrahiert nun auch
    Absätze aus SDT-Containern, in denen Word Verzeichnisse oft
versteckt. 2) Das
    Schlüsselwort "Inhalt" wurde hinzugefügt. 3) Verzeichnisse werden
nicht mehr
    nur gelöscht, sondern als intelligenter, vergleichbarer
Gesamt-Block
    ("Inhalt" / "Verzeichnis") ganz oben in den HTML-Report eingefügt.
4) Die
    Word-COM-Suche ist nun unanfällig für Zeilenumbrüche in
Überschriften.
```

Vergleicht zwei Word-Dokumente (.docx) auf Basis von Kapitelnummern
als

Fixpunkten und erzeugt einen eigenstaendigen HTML-Report.

"""

```
import argparse
import base64
import difflib
import glob
import hashlib
import html
import json
import os
import re
import shutil
import subprocess
import sys
import tempfile
import threading
import time
import zipfile
from datetime import datetime
from pathlib import Path

from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Pt, RGBColor, Twips
from docx.text.paragraph import Paragraph
from lxml import etree
```

SCRIPT_VERSION = "3.10"

REVIEW_SCHEMA_VERSION = "1.0"

```
REVIEW_STATUS_OPTIONS = [
    ("", "– nicht bewertet –"),
    ("accepted", "Accepted"),
    ("not_accepted", "Not accepted"),
    ("refinement_customer", "Refinement needed from customer"),
    ("internal_clarification", "Internal clarification needed"),
]
```

#

Kapitel-Erkennung

#


```

ANCHOR_TAB_RE = re.compile(r"^(\\d+(?:\\.\\d+)*[a-zA-Z]?)\\s*\\t\\s*(.*)$")
BARE_NUMBER_RE = re.compile(r"^(\\d+[a-zA-Z]?)$")
DOT_CONTINUATION_RE =
re.compile(r"^(\\.\\d+(?:\\.\\d+)*[a-zA-Z]?)\\s*\\t?\\s*(.*)$")
WEB_SAFE_IMAGE_TYPES = {"image/png", "image/jpeg", "image/jpg",
"image/gif", "image/bmp", "image/webp"}
W_NS = "http://schemas.openxmlformats.org/wordprocessingml/2006/main"

```

```

def _load_numbering_definitions(docx_path):
    try:
        with zipfile.ZipFile(docx_path) as z:
            if "word/numbering.xml" not in z.namelist():
                return {}, {}, {}
            xml_bytes = z.read("word/numbering.xml")
    except Exception:
        return {}, {}, {}

    try:
        root = etree.fromstring(xml_bytes)
    except Exception:
        return {}, {}, {}

    ns = {"w": W_NS}
    abstract_levels = {}
    style_linked = {}
    for abstract_el in root.findall("w:abstractNum", ns):
        abstract_id = abstract_el.get(f"{{{W_NS}}}abstractNumId")
        levels = {}
        for lvl in abstract_el.findall("w:lvl", ns):
            numfmt_el = lvl.find("w:numFmt", ns)
            numfmt = numfmt_el.get(f"{{{W_NS}}}val") if numfmt_el is
not None else "decimal"
            if numfmt != "decimal":
                continue
            ilvl = int(lvl.get(f"{{{W_NS}}}ilvl", "0"))
            start_el = lvl.find("w:start", ns)
            start_val = int(start_el.get(f"{{{W_NS}}}val", "1")) if
start_el is not None else 1
            levels[ilvl] = {"start": start_val}

            pstyle_el = lvl.find("w:pStyle", ns)
            if pstyle_el is not None:
                style_id = pstyle_el.get(f"{{{W_NS}}}val")
                if style_id:
                    style_linked[style_id] = {"ilvl": ilvl, "start":
start_val}

```

```

        if abstract_id is not None:
            abstract_levels[abstract_id] = levels

num_to_abstract = {}
for num_el in root.findall("w:num", ns):
    num_id = num_el.get(f"{{{W_NS}}}numId")
    abstract_ref = num_el.find("w:abstractNumId", ns)
    if num_id is not None and abstract_ref is not None:
        num_to_abstract[num_id] =
abstract_ref.get(f"{{{W_NS}}}val")

return num_to_abstract, abstract_levels, style_linked

def _load_style_numpr(docx_path):
    try:
        with zipfile.ZipFile(docx_path) as z:
            if "word/styles.xml" not in z.namelist():
                return {}
            xml_bytes = z.read("word/styles.xml")
    except Exception:
        return {}
    try:
        root = etree.fromstring(xml_bytes)
    except Exception:
        return {}

    ns = {"w": W_NS}
    result = {}
    for style_el in root.findall("w:style", ns):
        style_id = style_el.get(f"{{{W_NS}}}styleId")
        num_pr = style_el.find("w:pPr/w:numPr", ns)
        if style_id and num_pr is not None:
            num_id_el = num_pr.find("w:numId", ns)
            ilvl_el = num_pr.find("w:ilvl", ns)
            if num_id_el is not None:
                num_id = num_id_el.get(f"{{{W_NS}}}val")
                ilvl = int(ilvl_el.get(f"{{{W_NS}}}val", "0")) if
ilvl_el is not None else 0
                if num_id:
                    result[style_id] = (num_id, ilvl)
    return result

def _paragraph_numpr(paragraph):
    try:
        num_pr = paragraph._p.find(f"{{{W_NS}}}pPr/{{{{{W_NS}}}}numPr")
    except Exception:

```

```

        return None
    if num_pr is None:
        return None
    num_id_el = num_pr.find(f"{{{W_NS}}}{numId}")
    if num_id_el is None:
        return None
    num_id = num_id_el.get(f"{{{W_NS}}}{val}")
    ilvl_el = num_pr.find(f"{{{W_NS}}}{ilvl}")
    ilvl = int(ilvl_el.get(f"{{{W_NS}}}{val}", "0")) if ilvl_el is not
None else 0
    return (num_id, ilvl) if num_id else None

```

```

class _HeadingNumberer:
    def __init__(self, num_to_abstract, abstract_levels, style_numpr,
style_linked):
        self.num_to_abstract = num_to_abstract
        self.abstract_levels = abstract_levels
        self.style_numpr = style_numpr
        self.style_linked = style_linked
        self.counters = {}

    def _level_info(self, num_id, ilvl):
        abstract_id = self.num_to_abstract.get(num_id)
        if abstract_id is None:
            return None, None
        info = self.abstract_levels.get(abstract_id, {}).get(ilvl)
        return abstract_id, info

    def number_for_paragraph(self, paragraph):
        style_id = _style_id(paragraph)

        direct = _paragraph_numpr(paragraph)
        if direct is not None:
            num_id, ilvl = direct
            abstract_id, info = self._level_info(num_id, ilvl)
            if info is not None:
                return self._advance(abstract_id, ilvl, info["start"])

        if style_id and style_id in self.style_numpr:
            num_id, ilvl = self.style_numpr[style_id]
            abstract_id, info = self._level_info(num_id, ilvl)
            if info is not None:
                return self._advance(abstract_id, ilvl, info["start"])

        if style_id and style_id in self.style_linked:
            entry = self.style_linked[style_id]
            return self._advance(f"_stylelinked_{style_id}",

```

```

entry["ilvl"], entry["start"], use_style_key=True)

    return None

def _advance(self, counter_key, ilvl, start, use_style_key=False):
    counters = self.counters.setdefault(counter_key, {})
    if ilvl in counters:
        counters[ilvl] += 1
    else:
        counters[ilvl] = start
    for deeper in [l for l in counters if l > ilvl]:
        del counters[deeper]
    parts = [str(counters.get(l, 1)) for l in range(ilvl + 1)]
    return ".".join(parts)

def _style_id(paragraph):
    try:
        return paragraph.style.style_id
    except Exception:
        return None

def _extract_images(paragraph):
    images = []
    part = paragraph.part
    p_xml = paragraph._p

    blips = p_xml.findall("./" + qn("a:blip"))
    for blip in blips:
        rId = blip.get(qn("r:embed")) or blip.get(qn("r:link"))
        if not rId or rId not in part.rels:
            continue
        try:
            image_part = part.rels[rId].target_part
            blob = image_part.blob
            content_type = image_part.content_type
        except Exception:
            continue
        images.append({"hash": hashlib.sha1(blob).hexdigest(), "blob":
blob, "content_type": content_type})

    VML_NS = "urn:schemas-microsoft-com:vml"
    imagedatas = p_xml.findall(f"./{{{VML_NS}}}imagedata")
    for imgdata in imagedatas:
        rId = imgdata.get(qn("r:id"))
        if not rId or rId not in part.rels:
            continue

```

```

        try:
            image_part = part.rels[rId].target_part
            blob = image_part.blob
            content_type = image_part.content_type
        except Exception:
            continue
        h = hashlib.sha1(blob).hexdigest()
        if not any(im["hash"] == h for im in images):
            images.append({"hash": h, "blob": blob, "content_type":
content_type})

    return images

def _iter_paragraphs_including_sdts(doc):
    """Extrahiert alle Absätze, auch wenn diese in einem Structured
Document Tag (SDT)
    versteckt sind (wie es Word oft bei Inhaltsverzeichnissen macht).
Tabellen werden
    übersprungen, um die Konsistenz zur Standard-doc.paragraphs Logik
zu wahren."""
    paras = []
    def walk(element):
        for child in element.iterchildren():
            if child.tag == qn('w:p'):
                paras.append(Paragraph(child, doc._body))
            elif child.tag == qn('w:tbl'):
                pass # Tabelleninhalte nicht als Kapitelanfänge
werten
            else:
                walk(child)
    walk(doc.element.body)
    return paras

_TOC_HEADING_TEXTS = {
    "inhaltsverzeichnis", "inhalt", "table of contents", "contents",
    "abbildungsverzeichnis", "list of figures",
    "tabellenverzeichnis", "list of tables",
    "abkürzungsverzeichnis", "abkuerzungsverzeichnis", "list of
abbreviations",
    "glossar", "glossary",
}
_TOC_EXIT_TEXT_LEN = 200
# Erkennt typische TOC-Zeilen: Text, dann Tabulator oder viele Punkte,
dann Ziffer(n) am Ende
_TOC_LINE_RE = re.compile(r"(?:\.{5,}|\t)[ \t]*\d+[ \t]*$")

```

```

def extract_chapters(docx_path):
    doc = Document(docx_path)
    num_to_abstract, abstract_levels, style_linked =
_load_numbering_definitions(docx_path)
    style_numpr = _load_style_numpr(docx_path)
    numberer = _HeadingNumberer(num_to_abstract, abstract_levels,
style_numpr, style_linked)

    chapters = []
    current = None
    pending_numbers = []
    fallback_counter = 0
    current_para_idx = -1
    in_toc_section = False
    toc_section_para_count = 0
    _TOC_SECTION_MAX_PARAS = 1500

    def new_chapter(number, source="fallback"):
        nonlocal current, fallback_counter
        if number is None:
            fallback_counter += 1
            number = f"_{fallback_counter}"
        current = {"number": number, "title": "", "paragraphs": [],
"images": [],
                    "first_para_index": current_para_idx, "_source":
source}
        chapters.append(current)

    def append_text(text):
        nonlocal current
        if current is None:
            new_chapter(None)
        if text:
            current["paragraphs"].append(text)

    def append_images(imgs):
        nonlocal current
        if not imgs:
            return
        if current is None:
            new_chapter(None)
        current["images"].extend(imgs)

    # Nutzt unseren neuen SDT-kompatiblen Iterator
    for para_idx, para in
enumerate(_iter_paragraphs_including_sdt(doc)):
        current_para_idx = para_idx

```

```

text = para.text.strip()
imgs = _extract_images(para)

text_lower = text.strip().lower().rstrip(":")
is_toc_heading = text_lower in _TOC_HEADING_TEXTS
is_toc_line = bool(_TOC_LINE_RE.search(text))

# 1) Start eines Verzeichnisses erkannt
if is_toc_heading:
    in_toc_section = True
    toc_section_para_count = 0
    pending_numbers.clear()
    new_chapter(text.strip(), source="toc_heading")
    continue

if in_toc_section:
    toc_section_para_count += 1
    is_real_heading = False
    auto_number_probe = numberer.number_for_paragraph(para)

    # Reset, wenn wir eine echte TOC-Zeile sehen
    if is_toc_line:
        is_real_heading = False
        toc_section_para_count = 0
    else:
        if auto_number_probe is not None:
            is_real_heading = True
        elif ANCHOR_TAB_RE.match(text) or
DOT_CONTINUATION_RE.match(text) or BARE_NUMBER_RE.match(text):
            is_real_heading = True

    # Solange wir keine echte neue Überschrift sehen, sind wir
    noch im Inhaltsverzeichnis
    if not is_real_heading and len(text) < _TOC_EXIT_TEXT_LEN
and toc_section_para_count < 20:
        append_text(text)
        if imgs:
            append_images(imgs)
        continue

    # Echtes Kapitel beginnt -> Verzeichnis-Modus verlassen
    in_toc_section = False

# 2) Stray TOC line (z.B. Abbildungsverzeichnis, das direkt
folgt, aber keine eigene Überschrift hat)
if is_toc_line:
    if current is None:
        new_chapter("Verzeichnis", source="toc_heading")

```

```

        append_text(text)
    if imgs:
        append_images(imgs)
    continue

# 3) Normales Kapitel-Parsing
auto_number = numberer.number_for_paragraph(para)
if auto_number is not None:
    pending_numbers.clear()
    new_chapter(auto_number, source="heading_auto")
    if text:
        append_text(text)
elif not text:
    pass
else:
    m_tab = ANCHOR_TAB_RE.match(text)
    m_dot = DOT_CONTINUATION_RE.match(text)
    m_bare = BARE_NUMBER_RE.match(text)

    if m_tab:
        pending_numbers.clear()
        new_chapter(m_tab.group(1), source="tab_anchor")
        append_text(m_tab.group(2))
    elif m_dot and pending_numbers:
        base = pending_numbers.pop(0)
        new_chapter(f"{base}.{m_dot.group(1)}",
source="dot_continuation")
        append_text(m_dot.group(2))
    elif m_bare:
        pending_numbers.append(m_bare.group(1))
    elif pending_numbers:
        number = pending_numbers.pop(0)
        pending_numbers.clear()
        new_chapter(number, source="bare_fallback")
        append_text(text)
    else:
        append_text(text)

    if imgs:
        append_images(imgs)

for ch in chapters:
    ch["text"] = "\n".join(ch["paragraphs"])
    ch["level"] = 1

return chapters

```

```

def index_by_key(chapters):
    idx = {}
    counts = {}
    for ch in chapters:
        num = ch["number"]
        counts[num] = counts.get(num, 0) + 1
        key = num if counts[num] == 1 else f"{num}__{counts[num]}"
        ch["key"] = key
        idx[key] = ch
    return idx

#
-----
# Seiten-Erkennung ("Buchform") - OPTIONAL
#
-----

_WIN_DRIVES = ["C", "D", "E", "F"]

SOFFICE_CANDIDATES = [
    "soffice",
    "libreoffice",
    "/usr/bin/soffice",
    "/opt/libreoffice/program/soffice",
    "/Applications/LibreOffice.app/Contents/MacOS/soffice",
] + [
    fr"{drive}:\Program Files\LibreOffice\program\soffice.exe" for
drive in _WIN_DRIVES
] + [
    fr"{drive}:\Program Files (x86)\LibreOffice\program\soffice.exe"
for drive in _WIN_DRIVES
]

SOFFICE_GLOB_PATTERNS = [
    fr"{drive}:\Program Files\LibreOffice*\program\soffice.exe" for
drive in _WIN_DRIVES
] + [
    fr"{drive}:\Program Files (x86)\LibreOffice*\program\soffice.exe"
for drive in _WIN_DRIVES
] + [
    str(Path.home() /
"AppData/Local/Programs/LibreOffice*/program/soffice.exe"),
]

def find_soffice(explicit_path=None):

```

```

if explicit_path:
    return explicit_path if Path(explicit_path).exists() else None
env_path = os.environ.get("SOFFICE_PATH")
if env_path and Path(env_path).exists():
    return env_path
for candidate in SOFFICE_CANDIDATES:
    if Path(candidate).is_absolute():
        if Path(candidate).exists():
            return candidate
    else:
        found = shutil.which(candidate)
        if found:
            return found
for pattern in SOFFICE_GLOB_PATTERNS:
    matches = sorted(glob.glob(pattern), reverse=True)
    if matches:
        return matches[0]
return None

def render_page_texts(docx_path, soffice_path, timeout=90):
    try:
        import pdfplumber
    except ImportError:
        return None
    try:
        with tempfile.TemporaryDirectory() as tmp:
            subprocess.run(
                [soffice_path, "--headless", "--convert-to", "pdf",
                "--outdir", tmp, str(docx_path)],
                check=True, timeout=timeout, capture_output=True,
            )
            pdf_path = Path(tmp) / (Path(docx_path).stem + ".pdf")
            if not pdf_path.exists():
                return None
            with pdfplumber.open(pdf_path) as pdf:
                return [normalize_whitespace(p.extract_text() or "")
                for p in pdf.pages]
    except Exception:
        return None

def _detect_toc_end_page(page_texts, max_scan_pages=15,
min_toc_lines=2):
    last_toc_page = -1
    pattern = re.compile(r"(?:^|[\n]) [
\t]*(?:Abbildung|Figure|Table|Tabelle|[\d\.]+) [^\n]{2,200}?(?:\.{4,}|\
t|\s{3,})\d+[ \t]*(?=[\n]|$)", re.IGNORECASE)

    for i, text in enumerate(page_texts[:max_scan_pages]):

```

```

        matches = list(pattern.finditer(text))
        dot_matches = list(re.finditer(r"\.{10,}", text))

        if len(matches) >= min_toc_lines or len(dot_matches) >=
min_toc_lines:
            last_toc_page = i
        elif last_toc_page != -1 and i == last_toc_page + 1:
            if len(matches) > 0 or len(dot_matches) > 0:
                last_toc_page = i
        return last_toc_page

def assign_pages_to_chapters(chapters, page_texts, key_len=40):
    if not page_texts:
        for ch in chapters:
            ch["page"] = None
        return 0

    toc_end_page = _detect_toc_end_page(page_texts)
    page_idx = 0
    cursor = 0
    for ch in chapters:
        is_toc = ch.get("_source") == "toc_heading"
        if is_toc:
            key = ch["number"][:key_len]
        else:
            if toc_end_page >= 0 and page_idx <= toc_end_page:
                page_idx = toc_end_page + 1
                cursor = 0

            text_lines = [line.strip() for line in ch.get("text",
"").split('\n') if line.strip()]
            best_line = ""
            for line in text_lines[:3]:
                if len(line) > len(best_line):
                    best_line = line
            key = normalize_whitespace(best_line)[:key_len]

        if key:
            while True:
                if page_idx >= len(page_texts):
                    break
                pos = page_texts[page_idx].find(key, cursor)
                if pos != -1:
                    cursor = pos + len(key)
                    break
                page_idx += 1
            cursor = 0

```

```

        ch["page"] = page_idx + 1 if page_idx < len(page_texts) else
None
        return toc_end_page + 1 if toc_end_page >= 0 else 0

def find_word_com():
    if sys.platform != "win32":
        return False
    try:
        import win32com.client # noqa: F401
        return True
    except ImportError:
        return False

def assign_pages_via_word_com(chapters, docx_path, timeout=120):
    if sys.platform != "win32":
        return False, False
    try:
        import pywintypes
        import win32com.client
        import pythoncom
    except ImportError:
        return False, False

    if not chapters:
        return False, False

    word = None
    doc = None
    toc_detected = False
    try:
        pythoncom.CoInitialize()
        word = win32com.client.DispatchEx("Word.Application")

        if word.Documents.Count > 0:
            return False, False
        word.Visible = False
        word.DisplayAlerts = 0
        doc = word.Documents.Open(
            str(Path(docx_path).resolve()), ReadOnly=True,
AddToRecentFiles=False, Visible=False,
            ConfirmConversions=False,
        )
        try:
            doc.ActiveWindow.View.Type = 3
        except Exception:
            pass
        try:
            doc.Repaginate()

```

```

except Exception:
    pass
try:
    doc.ComputeStatistics(2)
except Exception:
    pass

WD_ACTIVE_END_PAGE_NUMBER = 3
WD_FIND_STOP = 0
FIND_KEY_LEN = 80
doc_end = doc.Content.End
cursor_start = 0
found_count = 0

toc_cursor_jump = 0
try:
    for toc in doc.TablesOfContents:
        if toc.Range.End > toc_cursor_jump:
            toc_cursor_jump = toc.Range.End
            toc_detected = True
    for tof in doc.TablesOfFigures:
        if tof.Range.End > toc_cursor_jump:
            toc_cursor_jump = tof.Range.End
            toc_detected = True
except Exception:
    pass

try:
    probe_text = doc.Content.Text[:30000]
    pattern = re.compile(r"(?:^|[\r\n])[\t]*(?:Abbildung|Figure|Table|Tabelle|[\d\.]*)[\r\n]{2,200}?(?:\.{5,}|\t)[\t]*\d+[\t]*(?=[\r\n]|$)", re.IGNORECASE)
    toc_matches = list(pattern.finditer(probe_text))
    if len(toc_matches) >= 2:
        last_end = toc_matches[-1].end()
        if last_end > toc_cursor_jump:
            toc_cursor_jump = last_end
            toc_detected = True

    dot_matches = list(re.finditer(r"\.{10,}", probe_text))
    if len(dot_matches) >= 3:
        last_dot_end = dot_matches[-1].end()
        if last_dot_end > toc_cursor_jump:
            toc_cursor_jump = last_dot_end
            toc_detected = True
except Exception:
    pass

```

```

for ch in chapters:
    is_toc = ch.get("_source") == "toc_heading"
    if is_toc:
        key = ch["number"][:FIND_KEY_LEN]
    else:
        if cursor_start < toc_cursor_jump:
            cursor_start = toc_cursor_jump

        text_lines = [line.strip() for line in ch.get("text",
"".split('\n')) if line.strip()]
        best_line = ""
        for line in text_lines[:3]:
            if len(line) > len(best_line):
                best_line = line
        key = normalize_whitespace(best_line)[:FIND_KEY_LEN]

    if not key or cursor_start >= doc_end:
        ch["page"] = None
        continue
    try:
        rng = doc.Range(cursor_start, doc_end)
        f = rng.Find
        f.ClearFormatting()
        f.Text = key
        f.Forward = True
        f.Wrap = WD_FIND_STOP
        f.MatchCase = False
        f.MatchWholeWord = False
        f.MatchWildcards = False
        found = f.Execute()
        if found:
            ch["page"] =
rng.Information(WD_ACTIVE_END_PAGE_NUMBER)
            cursor_start = rng.End
            found_count += 1
        else:
            ch["page"] = None
    except Exception:
        ch["page"] = None

    return found_count > 0, toc_detected
except Exception:
    return False, False
finally:
    try:
        if doc is not None:
            doc.Close(SaveChanges=False)
    except Exception:

```

```

        pass
    try:
        if word is not None:
            if word.Documents.Count == 0:
                word.Quit()
    except Exception:
        pass
    try:
        pythoncom.CoUninitialize()
    except Exception:
        pass

def attach_pages(chapters_a, chapters_b, path_a, path_b,
soffice_path=None, word_timeout_s=45):
    toc_info = {"detected": False, "pages_skipped_a": None,
"pages_skipped_b": None}

    if find_word_com():
        status_a, result_a, _, _ =
_run_with_timeout(assign_pages_via_word_com, word_timeout_s,
chapters_a, path_a)
        ok_a, toc_a = (result_a if status_a == "ok" and result_a else
(False, False))
        ok_b, toc_b = False, False
        if ok_a:
            status_b, result_b, _, _ =
_run_with_timeout(assign_pages_via_word_com, word_timeout_s,
chapters_b, path_b)
            ok_b, toc_b = (result_b if status_b == "ok" and result_b
else (False, False))
        if ok_a and ok_b:
            toc_info["detected"] = bool(toc_a or toc_b)
            return "word_com", toc_info
        for ch in chapters_a:
            ch.pop("page", None)
        for ch in chapters_b:
            ch.pop("page", None)

soffice_path = find_soffice(soffice_path)
if soffice_path:
    pages_a = render_page_texts(path_a, soffice_path)
    pages_b = render_page_texts(path_b, soffice_path)
    skipped_a = assign_pages_to_chapters(chapters_a, pages_a)
    skipped_b = assign_pages_to_chapters(chapters_b, pages_b)
    if pages_a and pages_b:
        toc_info["detected"] = bool(skipped_a or skipped_b)
        toc_info["pages_skipped_a"] = skipped_a

```

```

        toc_info["pages_skipped_b"] = skipped_b
        return "libreoffice", toc_info

    for ch in chapters_a:
        ch["page"] = None
    for ch in chapters_b:
        ch["page"] = None
    return None, toc_info

#
-----
# Diff-Logik
#
-----

_TOKEN_RE = re.compile(r"\s+|\S+")

def _tokenize(text):
    return _TOKEN_RE.findall(text)

def render_diff_pair(text_a, text_b):
    tokens_a = _tokenize(text_a)
    tokens_b = _tokenize(text_b)
    sm = difflib.SequenceMatcher(None, tokens_a, tokens_b,
    autojunk=False)

    left_parts, right_parts = [], []
    for tag, i1, i2, j1, j2 in sm.get_opcodes():
        seg_a = html.escape("".join(tokens_a[i1:i2]))
        seg_b = html.escape("".join(tokens_b[j1:j2]))
        if tag == "equal":
            left_parts.append(seg_a)
            right_parts.append(seg_b)
        elif tag == "delete":
            left_parts.append(f'<span class="del">{seg_a}</span>')
        elif tag == "insert":
            right_parts.append(f'<span class="ins">{seg_b}</span>')
        elif tag == "replace":
            left_parts.append(f'<span class="del">{seg_a}</span>')
            right_parts.append(f'<span class="ins">{seg_b}</span>')

    return (
        "".join(left_parts).replace("\n", "<br>"),
        "".join(right_parts).replace("\n", "<br>"),
    )

```

```

def plain_html(text):
    return html.escape(text).replace("\n", "<br>")

def image_hash_set(chapter):
    return sorted(img["hash"] for img in chapter.get("images", []))

def classify(ch_a, ch_b):
    images_changed = image_hash_set(ch_a) != image_hash_set(ch_b)
    if ch_a["text"] == ch_b["text"] and ch_a["title"] == ch_b["title"]
and not images_changed:
        return "unchanged", 1.0, False
    ratio = difflib.SequenceMatcher(None, ch_a["text"], ch_b["text"],
autojunk=False).ratio()
    return "changed", ratio, images_changed

def natural_sort_key(number, order_index):
    # Pinnt Verzeichnisse fest nach ganz oben an den Dokumentanfang
    if number.lower() in _TOC_HEADING_TEXTS or number ==
"Verzeichnis":
        return (-1, order_index, ())
    if number.startswith("_"):
        return (1, order_index, ())
    parts = number.split(".")
    try:
        parsed = tuple(int(p) for p in parts)
    except ValueError:
        return (1, order_index, ())
    return (0, order_index, parsed)

def _similarity(a, b):
    if not a or not b:
        return 0.0
    return difflib.SequenceMatcher(None, a, b, autojunk=False).ratio()

def fallback_match_unnumbered(deleted_only, new_only, threshold=0.6,
time_budget_s=5.0):
    candidates_a = [c for c in deleted_only if
c["number"].startswith("_") or c.get("_source") == "toc_heading"]
    candidates_b = [c for c in new_only if c["number"].startswith("_")
or c.get("_source") == "toc_heading"]

    pairs = []
    used_b_keys = set()
    start = time.perf_counter()
    for ca in candidates_a:
        if time.perf_counter() - start > time_budget_s:
            break
        best, best_score = None, 0.0

```

```

    ca_title, ca_text = ca["title"], ca["text"]
    ca_len = len(ca_text)
    for cb in candidates_b:
        if cb["key"] in used_b_keys:
            continue
        score = _similarity(ca_title, cb["title"])
        cb_text = cb["text"]
        total_len = ca_len + len(cb_text)
        if total_len and (2 * min(ca_len, len(cb_text)) /
total_len) >= threshold:
            sm = difflib.SequenceMatcher(None, ca_text, cb_text,
autojunk=False)
            if sm.quick_ratio() >= threshold:
                score = max(score, sm.ratio())
            if score > best_score:
                best_score, best = score, cb
        if best is not None and best_score >= threshold:
            used_b_keys.add(best["key"])
            pairs.append((ca, best, best_score))
    return pairs

def normalize_whitespace(text):
    return re.sub(r"\s+", " ", text).strip()

def _normalize_chapters_for_comparison(chapters):
    out = []
    for ch in chapters:
        new_ch = dict(ch)
        new_ch["text"] = normalize_whitespace(ch["text"])
        new_ch["title"] = normalize_whitespace(ch["title"]) if
ch["title"] else ch["title"]
        out.append(new_ch)
    return out

def build_comparison(chapters_a, chapters_b, ignore_linebreaks=True):
    if ignore_linebreaks:
        chapters_a = _normalize_chapters_for_comparison(chapters_a)
        chapters_b = _normalize_chapters_for_comparison(chapters_b)

    idx_a = index_by_key(chapters_a)
    idx_b = index_by_key(chapters_b)
    order_a = {ch["key"]: i for i, ch in enumerate(chapters_a)}
    order_b = {ch["key"]: i for i, ch in enumerate(chapters_b)}

    all_keys = list(dict.fromkeys(list(idx_a.keys()) +
list(idx_b.keys()))))
    rows = []
    deleted_only, new_only = [], []

```

```

for key in all_keys:
    ca = idx_a.get(key)
    cb = idx_b.get(key)

    if ca and cb:
        status, ratio, images_changed = classify(ca, cb)
        if status == "changed":
            html_a, html_b = render_diff_pair(ca["text"],
cb["text"])
        else:
            html_a, html_b = plain_html(ca["text"]),
plain_html(cb["text"])
        rows.append({
            "key": key, "number": ca["number"], "status": status,
"ratio": ratio,
            "title_a": ca["title"], "title_b": cb["title"],
            "html_a": html_a, "html_b": html_b,
            "images_a": ca.get("images", []), "images_b":
cb.get("images", []),
            "images_changed": images_changed,
            "page_a": ca.get("page"), "page_b": cb.get("page"),
            "text_a_raw": ca["text"], "text_b_raw": cb["text"],
        })
    elif ca and not cb:
        deleted_only.append(ca)
    else:
        new_only.append(cb)

fallback_pairs = fallback_match_unnumbered(deleted_only, new_only)
matched_a_keys = {ca["key"] for ca, _, _ in fallback_pairs}
matched_b_keys = {cb["key"] for _, cb, _ in fallback_pairs}

for ca, cb, score in fallback_pairs:
    status, ratio, images_changed = classify(ca, cb)
    if status == "changed":
        html_a, html_b = render_diff_pair(ca["text"], cb["text"])
    else:
        html_a, html_b = plain_html(ca["text"]),
plain_html(cb["text"])
    number = ca["number"] if not ca["number"].startswith("_") else
cb["number"]
    rows.append({
        "key": ca["key"], "number": number, "status": status,
"ratio": ratio,
        "title_a": ca["title"], "title_b": cb["title"],
        "html_a": html_a, "html_b": html_b,
        "images_a": ca.get("images", []), "images_b":

```

```

cb.get("images", []),
    "images_changed": images_changed,
    "page_a": ca.get("page"), "page_b": cb.get("page"),
    "text_a_raw": ca["text"], "text_b_raw": cb["text"],
    })
    order_a[ca["key"]] = order_a.get(ca["key"],
order_a.get(cb["key"], 0))

for ca in deleted_only:
    if ca["key"] in matched_a_keys:
        continue
    rows.append({
        "key": ca["key"], "number": ca["number"], "status":
"deleted", "ratio": 0.0,
        "title_a": ca["title"], "title_b": None,
        "html_a": plain_html(ca["text"]), "html_b": None,
        "images_a": ca.get("images", []), "images_b": [],
        "images_changed": bool(ca.get("images")),
        "page_a": ca.get("page"), "page_b": None,
        "text_a_raw": ca["text"], "text_b_raw": "",
    })

for cb in new_only:
    if cb["key"] in matched_b_keys:
        continue
    rows.append({
        "key": cb["key"], "number": cb["number"], "status": "new",
"ratio": 0.0,
        "title_a": None, "title_b": cb["title"],
        "html_a": None, "html_b": plain_html(cb["text"]),
        "images_a": [], "images_b": cb.get("images", []),
        "images_changed": bool(cb.get("images")),
        "page_a": None, "page_b": cb.get("page"),
        "text_a_raw": "", "text_b_raw": cb["text"],
    })

def sort_key(row):
    oi = order_a.get(row["key"])
    if oi is None:
        oi = 10_000_000 + order_b.get(row["key"], 0)
    return natural_sort_key(row["number"], oi)

rows.sort(key=sort_key)
return rows

MOVE_MIN_LEN = 30
MOVE_THRESHOLD = 0.55

```

```

def _diff_removed_added(text_a, text_b):
    tokens_a = _tokenize(text_a)
    tokens_b = _tokenize(text_b)
    sm = difflib.SequenceMatcher(None, tokens_a, tokens_b,
    autojunk=False)
    removed, added = [], []
    for tag, i1, i2, j1, j2 in sm.get_opcodes():
        if tag in ("delete", "replace"):
            removed.append("".join(tokens_a[i1:i2]))
        if tag in ("insert", "replace"):
            added.append("".join(tokens_b[j1:j2]))
    return normalize_whitespace("".join(removed)),
    normalize_whitespace("".join(added))

def detect_possible_moves(rows, min_len=MOVE_MIN_LEN,
threshold=MOVE_THRESHOLD, time_budget_s=8.0):
    row_by_key = {r["key"]: r for r in rows}
    removed_pool, added_pool = [], []

    for r in rows:
        status = r["status"]
        text_a = r.get("text_a_raw", "") or ""
        text_b = r.get("text_b_raw", "") or ""
        if status == "deleted":
            if len(text_a) >= min_len:
                removed_pool.append((r["key"], r["number"], text_a))
        elif status == "new":
            if len(text_b) >= min_len:
                added_pool.append((r["key"], r["number"], text_b))
        elif status == "changed":
            removed_text, added_text = _diff_removed_added(text_a,
text_b)
            if len(removed_text) >= min_len:
                removed_pool.append((r["key"], r["number"],
removed_text))
            if len(added_text) >= min_len:
                added_pool.append((r["key"], r["number"], added_text))

    added_pool_len = [(akey, anum, atext, len(atext)) for akey, anum,
atext in added_pool]

    moves = []
    complete = True
    start = time.perf_counter()
    for rkey, rnum, rtext in removed_pool:
        if time.perf_counter() - start > time_budget_s:
            complete = False

```

```

        break
    rlen = len(rtext)
    best_key, best_num, best_score = None, None, 0.0
    for akey, anum, atext, alen in added_pool_len:
        if akey == rkey:
            continue
        total_len = rlen + alen
        if total_len == 0 or (2 * min(rlen, alen) / total_len) <
threshold:
            continue
        sm = difflib.SequenceMatcher(None, rtext, atext,
autojunk=False)
        if sm.quick_ratio() < threshold or sm.quick_ratio() <=
best_score:
            continue
        score = sm.ratio()
        if score > best_score:
            best_key, best_num, best_score = akey, anum, score
    if best_key is not None and best_score >= threshold:
        moves.append({
            "from_key": rkey, "from_number": rnum,
            "to_key": best_key, "to_number": best_num,
            "score": best_score,
        })
        row_by_key[rkey].setdefault("moves", []).append(
            {"direction": "to", "other_number": best_num, "score":
best_score}
        )
        row_by_key[best_key].setdefault("moves", []).append(
            {"direction": "from", "other_number": rnum, "score":
best_score}
        )
    return moves, complete

def compute_stats(chapters_a, chapters_b, rows):
    return {
        "total_a": len(chapters_a),
        "total_b": len(chapters_b),
        "unchanged": sum(1 for r in rows if r["status"] ==
"unchanged"),
        "changed": sum(1 for r in rows if r["status"] == "changed"),
        "new": sum(1 for r in rows if r["status"] == "new"),
        "deleted": sum(1 for r in rows if r["status"] == "deleted"),
    }

#
-----
-----

```

```

# HTML-Report
#
-----
-----

STATUS_LABEL = {
    "unchanged": "Unverändert", "changed": "Geändert", "new": "Neu",
    "deleted": "Gelöscht",
}
STATUS_ICON = {
    "unchanged": "\u2713", "changed": "\u270E", "new": "\u271A",
    "deleted": "\u2716",
}
STATUS_LABEL_SHORT = STATUS_LABEL
STATUS_COLORS_DOCX = {
    "unchanged": {"rgb": "16A34A", "fill": "DCFCE7"},
    "changed": {"rgb": "C2410C", "fill": "FFEDD5"},
    "new": {"rgb": "1D4ED8", "fill": "DBEAFE"},
    "deleted": {"rgb": "B91C1C", "fill": "FEE2E2"},
}

def _plain_preview(html_text, max_len=220):
    if not html_text:
        return ""
    plain = re.sub(r"<[^>]+>", "", html_text)
    plain = html.unescape(plain).strip()
    plain = re.sub(r"\s+", " ", plain)
    if len(plain) > max_len:
        plain = plain[:max_len].rsplit(" ", 1)[0] + " ..."
    return plain

def _set_cell_shading(cell, fill_hex):
    tcPr = cell._tc.get_or_add_tcPr()
    shd = XmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), fill_hex)
    tcPr.append(shd)

def _set_col_widths(table, widths_dxa):
    table.autofit = False
    for row in table.rows:
        for cell, width in zip(row.cells, widths_dxa):
            cell.width = Twips(width)
    for col, width in zip(table.columns, widths_dxa):
        col.width = Twips(width)

def generate_docx_report(rows, stats, name_a, name_b, output_path,

```

```

meta_a=None, meta_b=None, reviews=None):
    doc = Document()
    doc.add_heading("Kapitelvergleich", level=0)
    meta_a = meta_a or {"name": name_a, "modified": ""}
    meta_b = meta_b or {"name": name_b, "modified": ""}
    p = doc.add_paragraph()
    p.add_run(f"Dokument A: {meta_a['name']}").bold = True
    if meta_a.get("modified"):
        p.add_run(f"    (Stand: {meta_a['modified']})")
    p2 = doc.add_paragraph()
    p2.add_run(f"Dokument B: {meta_b['name']}").bold = True
    if meta_b.get("modified"):
        p2.add_run(f"    (Stand: {meta_b['modified']})")
    doc.add_paragraph(
        f"Erzeugt am {datetime.now().strftime('%d.%m.%Y %H:%M')} · "
        f"docx_chapter_compare.py Version {SCRIPT_VERSION}"
    ).runs[0].font.size = Pt(9)

    doc.add_heading("Zusammenfassung", level=1)
    summary = doc.add_table(rows=2, cols=6)
    summary.style = "Light Grid Accent 1"
    labels = ["Kapitel A", "Kapitel B", "Unverändert", "Geändert",
"Neu", "Gelöscht"]
    values = [stats["total_a"], stats["total_b"], stats["unchanged"],
stats["changed"], stats["new"], stats["deleted"]]
    for col, label in enumerate(labels):
        summary.cell(0, col).text = label
        summary.cell(0, col).paragraphs[0].runs[0].bold = True
    for col, value in enumerate(values):
        cell = summary.cell(1, col)
        cell.text = str(value)
        cell.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
        status_key = ["total_a", "total_b", "unchanged", "changed",
"new", "deleted"][col]
        if status_key in STATUS_COLORS_DOCX:
            colors = STATUS_COLORS_DOCX[status_key]
            _set_cell_shading(cell, colors["fill"])
            run = cell.paragraphs[0].runs[0]
            run.font.color.rgb = RGBColor.from_string(colors["rgb"])
            run.bold = True

    changed_rows = [r for r in rows if r["status"] != "unchanged"]

    doc.add_heading(f"Änderungen im Detail ({len(changed_rows)} )",
level=1)
    if not changed_rows:
        doc.add_paragraph("Keine Änderungen gefunden - beide Dokumente
sind inhaltlich identisch.")

```

```

else:
    has_reviews = bool(reviews)
    col_count = 5 if has_reviews else 4
    table = doc.add_table(rows=1, cols=col_count)
    table.style = "Light Grid Accent 1"
    headers = ["Nr.", "Status", f"Dokument A", f"Dokument B"]
    if has_reviews:
        headers.append("Review")
    for col, h in enumerate(headers):
        cell = table.cell(0, col)
        cell.text = h
        cell.paragraphs[0].runs[0].bold = True
    widths = [900, 1300, 3400, 3400] + ([2300] if has_reviews else
[])
    _set_col_widths(table, widths)

    for r in changed_rows:
        row_cells = table.add_row().cells
        row_cells[0].text = r["number"]
        status_cell = row_cells[1]
        status_cell.text = STATUS_LABEL_SHORT[r["status"]]
        colors = STATUS_COLORS_DOCX[r["status"]]
        _set_cell_shading(status_cell, colors["fill"])
        status_run = status_cell.paragraphs[0].runs[0]
        status_run.font.color.rgb =
RGBColor.from_string(colors["rgb"])
        status_run.bold = True
        if r["status"] == "changed":
            status_cell.add_paragraph(f"{int(r['ratio'] * 100)}%
gleich").runs[0].font.size = Pt(8)

        row_cells[2].text = _plain_preview(r.get("html_a")) or "-"
        row_cells[3].text = _plain_preview(r.get("html_b")) or "-"

        if has_reviews:
            entry = reviews.get(r["key"], {})
            status_val = entry.get("status", "")
            comment_val = entry.get("comment", "")
            label = dict(REVIEW_STATUS_OPTIONS).get(status_val, "–
nicht bewertet –")
            review_cell = row_cells[4]
            review_cell.text = label
            if comment_val:
                review_cell.add_paragraph(comment_val).runs[0].font.size = Pt(8)

            for cell in row_cells:
                for para in cell.paragraphs:

```

```

        for run in para.runs:
            if run.font.size is None:
                run.font.size = Pt(9)

    doc.add_heading(f"Vollständige Kapitelliste ({len(rows)} )",
level=1)
    overview = doc.add_table(rows=1, cols=3)
    overview.style = "Light List Accent 1"
    for col, h in enumerate(["Nr.", "Status", "Auszug"]):
        cell = overview.cell(0, col)
        cell.text = h
        cell.paragraphs[0].runs[0].bold = True
    _set_col_widths(overview, [900, 1600, 6500])
    for r in rows:
        row_cells = overview.add_row().cells
        row_cells[0].text = r["number"]
        status_cell = row_cells[1]
        status_cell.text = STATUS_LABEL_SHORT[r["status"]]
        run = status_cell.paragraphs[0].runs[0]
        run.font.color.rgb =
RGBColor.from_string(STATUS_COLORS_DOCX[r["status"]]["rgb"])
        preview = _plain_preview(r.get("html_b") or r.get("html_a"),
max_len=110)
        row_cells[2].text = preview
        for cell in row_cells:
            for para in cell.paragraphs:
                for run in para.runs:
                    run.font.size = Pt(9)

    doc.save(output_path)

def _safe_id(key):
    return re.sub(r"^[a-zA-Z0-9_-]", "_", key)

def doc_metadata(path):
    path = Path(path)
    try:
        modified =
datetime.fromtimestamp(path.stat().st_mtime).isoformat(timespec="secon
ds")
    except OSError:
        modified = ""
    return {"name": path.name, "modified": modified}

def _preview_title(title, body_html, max_len=70):
    if title:
        return title
    if not body_html:

```

```

        return ""
    plain = re.sub(r"<[^>]+>", "", body_html).replace("&", "&")
    plain = html.unescape(plain).strip()
    if len(plain) > max_len:
        plain = plain[:max_len].rsplit(" ", 1)[0] + " ..."
    return plain

def _images_html(images):
    if not images:
        return ""
    parts = ['<div class="img-gallery">']
    for img in images:
        ct = img.get("content_type", "")
        if ct in WEB_SAFE_IMAGE_TYPES:
            b64 = base64.b64encode(img["blob"]).decode("ascii")
            parts.append(f'')
        else:
            size_kb = max(1, len(img["blob"]) // 1024)
            label = ct or "unbekanntes Format"
            parts.append(f'<div class="img-placeholder" title="Keine
Browser-Vorschau moeglich"> {html.escape(label)} ({size_kb}
KB)</div>')
    parts.append("</div>")
    return "".join(parts)

def _page_classes(page, first, last):
    if page is None:
        return ""
    band = "pageband-even" if page % 2 == 0 else "pageband-odd"
    if first and last:
        grp = "pg-both"
    elif first:
        grp = "pg-first"
    elif last:
        grp = "pg-last"
    else:
        grp = "pg-mid"
    return f" {band} pg-grouped {grp}"

def _moves_html(moves_list):
    if not moves_list:
        return ""
    parts = []
    for m in moves_list:
        pct = int(m["score"] * 100)
        arrow = "→" if m["direction"] == "to" else "←"
        verb = "evtl. verschoben nach" if m["direction"] == "to" else

```

```

"evtl. hierher verschoben von"
    parts.append(f'<div class="move-note"> {verb} Kapitel
{html.escape(m["other_number"])} ({pct}% ähnlich) {arrow}</div>')
    return "".join(parts)

def _cell(number, title, body_html, images, side, status, page=None,
first_in_group=False, last_in_group=False, moves=None):
    page_class = _page_classes(page, first_in_group, last_in_group)
    page_tag = f'<div class="page-tag"> Seite {page}</div>' if (page
is not None and first_in_group) else ""

    if body_html is None:
        inner = (
            f'<div class="cell cell-empty cell-{{side}}{{page_class}}"
data-status="{{status}}">'
            f'<span class="empty-hint">– kein Kapitel
{html.escape(number)} in diesem Dokument –</span>'
            f"</div>"
        )
    else:
        display_title = _preview_title(title, body_html)
        header = f'<span class="chnum">{html.escape(number)}</span>
<span class="chttitle">{html.escape(display_title)}</span>'
        inner = (
            f'<details class="cell cell-{{side}}{{page_class}}"
data-status="{{status}}" {"open" if status != "unchanged" else ""}>'
            f"<summary>{header}</summary>"
            f'<div class="chbody">{body_html}</div>'
            f"{{_images_html(images)}} "
            f"{{_moves_html(moves)}} "
            f"</details>"
        )
    return f'<div class="cell-slot">{page_tag}{inner}</div>'

def _compute_group_flags(page_seq):
    n = len(page_seq)
    first_flags, last_flags = [False] * n, [False] * n
    next_real = [None] * n
    upcoming = None
    for i in range(n - 1, -1, -1):
        next_real[i] = upcoming
        if page_seq[i] is not None:
            upcoming = page_seq[i]
    last_real = None
    for i, p in enumerate(page_seq):
        if p is None:
            continue
        first_flags[i] = (p != last_real)

```

```

        last_flags[i] = (p != next_real[i])
        last_real = p
    return first_flags, last_flags

def render_html(rows, stats, name_a, name_b, ignore_linebreaks=True,
meta_a=None, meta_b=None,
                pages_method=None, diagnostics=None, moves=None,
moves_complete=True, toc_info=None):
    meta_a = meta_a or {"name": name_a, "modified": ""}
    meta_b = meta_b or {"name": name_b, "modified": ""}
    review_options_html = "".join(f'<option
value="{v}">{html.escape(label)}</option>' for v, label in
REVIEW_STATUS_OPTIONS)
    pages_a_seq = [r.get("page_a") for r in rows]
    pages_b_seq = [r.get("page_b") for r in rows]
    first_a, last_a = _compute_group_flags(pages_a_seq)
    first_b, last_b = _compute_group_flags(pages_b_seq)

    row_html = []
    for idx, r in enumerate(rows):
        status = r["status"]
        page_a, page_b = r.get("page_a"), r.get("page_b")
        row_moves = r.get("moves", [])
        left_moves = [m for m in row_moves if m["direction"] == "to"]
        right_moves = [m for m in row_moves if m["direction"] ==
"from"]
        left = _cell(r["number"], r["title_a"], r["html_a"],
r.get("images_a", []), "left", status,
                    page=page_a, first_in_group=first_a[idx],
last_in_group=last_a[idx], moves=left_moves)
        right = _cell(r["number"], r["title_b"], r["html_b"],
r.get("images_b", []), "right", status,
                    page=page_b, first_in_group=first_b[idx],
last_in_group=last_b[idx], moves=right_moves)
        pct = f'{int(r["ratio"] * 100)}%' if status == "changed" else
""
        img_badge = '<span class="conn-img-badge" title="Grafik
geändert"><img alt="icon" data-bbox="225 725 245 740"/></span>' if r.get("images_changed") else ""
        spine_class = " has-page-spine" if (page_a is not None or
page_b is not None) else ""
        connector = (
            f'<div class="connector conn-{status}{spine_class}">'
            f'<span class="conn-icon">{STATUS_ICON[status]}</span>'
            f'{img_badge}'
            f'<span class="conn-pct">{pct}</span>'
            f"</div>"
        )
        safe_id = _safe_id(r["key"])

```

```

        review_box = f"""
        <div class="review-box"
data-review-key="{html.escape(r['key'], quote=True)}">
        <label for="rs-{safe_id}">Review:</label>
        <select id="rs-{safe_id}" class="review-select"
onchange="onReviewChange('{safe_id}')">{review_options_html}</select>
        <input type="text" id="rc-{safe_id}" class="review-comment"
placeholder="Kommentar..." oninput="onReviewChange('{safe_id}')">
        </div>"""
        manual_link_box = f"""
        <div class="manual-link-box"
data-link-key="{html.escape(r['key'], quote=True)}">
        <div class="ml-row">
        <label for="ml-{safe_id}">🔗 Manuell verknüpfen
mit:</label>
        <input type="text" id="ml-{safe_id}"
class="manual-link-input" list="chapter-datalist"
placeholder="Kapitelnummer eingeben..."
oninput="onManualLinkChange('{safe_id}')"
autocomplete="off">
        <button type="button" class="ml-btn"
onclick="jumpToManualLink('{safe_id}')" title="Zur verknüpften Zeile
springen">↗</button>
        <button type="button" class="ml-btn ml-clear"
onclick="clearManualLink('{safe_id}')" title="Verknüpfung
zurücknehmen">X</button>
        </div>
        <div class="link-diff" id="ld-{safe_id}"></div>
        </div>"""
        continues_group = idx > 0 and not first_a[idx] and not
first_b[idx]
        wrapper_class = "row-wrapper continues-group" if
continues_group else "row-wrapper"
        row_html.append(
            f'<div class="{wrapper_class}" data-status="{status}">'
            f'<div class="grid-row row-{status}"
data-status="{status}">{left}{connector}{right}</div>'
            f"{review_box}"
            f"{manual_link_box}"
            f"</div>"
        )

    stats_html = f"""
    <div class="stat stat-total"><div
class="stat-num">{stats['total_a']}</div><div
class="stat-label">Kapitel A</div></div>
    <div class="stat stat-total"><div
class="stat-num">{stats['total_b']}</div><div>

```

```

class="stat-label">Kapitel B</div></div>
    <div class="stat stat-unchanged"><div
class="stat-num">{stats['unchanged']}</div><div
class="stat-label">Unverändert</div></div>
    <div class="stat stat-changed"><div
class="stat-num">{stats['changed']}</div><div
class="stat-label">Geändert</div></div>
    <div class="stat stat-new"><div
class="stat-num">{stats['new']}</div><div
class="stat-label">Neu</div></div>
    <div class="stat stat-deleted"><div
class="stat-num">{stats['deleted']}</div><div
class="stat-label">Gelöscht</div></div>
"""
    linebreak_note = "Zeilenumbrüche werden beim Vergleich ignoriert
(nur Wortinhalt zählt)." if ignore_linebreaks else "Zeilenumbrüche
werden strikt mitverglichen (inkl. Absatzgrenzen)."
    if pages_method == "word_com":
        page_note = "📄 Kapitel sind nach der von MS Word berechneten
Seite gruppiert (exakt, per COM-Automation)."
    elif pages_method == "libreoffice":
        page_note = "📄 Kapitel sind nach gerendeter Seite gruppiert
(via LibreOffice, Näherung)."
    elif pages_method == "unavailable":
        page_note = "📄 Seiten-Gruppierung nicht verfügbar (weder MS
Word/COM noch LibreOffice gefunden, oder fehlerhaft)."
    else:
        page_note = ""

    toc_note = ""
    if toc_info and toc_info.get("detected"):
        parts = []
        if toc_info.get("pages_skipped_a"):
            parts.append(f"Dokument A: {toc_info['pages_skipped_a']}
Seite(n)")
        if toc_info.get("pages_skipped_b"):
            parts.append(f"Dokument B: {toc_info['pages_skipped_b']}
Seite(n)")
        detail = f" ({', '.join(parts)} )" if parts else ""
        toc_note = f"📖 Verzeichnis am Dokumentanfang erkannt und im
Report gebündelt{detail}."

    diagnostics_html = ""
    if diagnostics:
        diag_items = "".join(f"<li>{html.escape(line)}</li>" for line
in diagnostics)
        diag_open = "open" if pages_method == "unavailable" else ""
        diagnostics_html = f"""

```

```

<details class="diagnostics-box" {diag_open}>
  <summary>🔧 Preflight-Diagnose (Seiten-Gruppierung) </summary>
  <ul>{diag_items}</ul>
</details>"""

moves_summary_html = ""
if moves:
    move_items = "".join(
        f"<li>Kapitel {html.escape(m['from_number'])} → Kapitel
{html.escape(m['to_number'])} "
        f"({int(m['score'] * 100)}% ähnlich)</li>"
        for m in moves
    )
    incomplete_note = (
        "<p style=\"color:#b45309;\">⚠ Zeitbudget ausgeschöpft -
Liste unvollständig.</p>"
        if not moves_complete else ""
    )
    moves_summary_html = f"""
<details class="moves-summary" open>
  <summary>🔍 Mögliche Verschiebungen erkannt
({len(moves)})</summary>
  {incomplete_note}<ul>{move_items}</ul>
</details>"""
    elif moves is not None and not moves_complete:
        moves_summary_html = """
<details class="moves-summary" open>
  <summary>🔍 Verschiebungs-Erkennung unvollständig</summary>
  <p style="color:#b45309;\">⚠ Zeitbudget ausgeschöpft, bevor alle
Paare geprüft wurden.</p>
</details>"""

doc_meta_json = json.dumps({"a": meta_a, "b": meta_b},
ensure_ascii=False)
script_version_json = json.dumps(SCRIPT_VERSION)
review_schema_json = json.dumps(REVIEW_SCHEMA_VERSION)
chapter_keys_json = json.dumps({r["key"]: r["number"] for r in
rows}, ensure_ascii=False)
safe_id_to_key_json = json.dumps({_safe_id(r["key"]): r["key"] for
r in rows}, ensure_ascii=False)
row_texts_json = json.dumps(
    {r["key"]: {"a": r.get("text_a_raw") or "", "b":
r.get("text_b_raw") or ""} for r in rows},
    ensure_ascii=False,
)

return f"""<!DOCTYPE html>
<html lang="de">

```

```

<head>
<meta charset="utf-8">
<title>Dokumentvergleich: {html.escape(name_a)} vs
{html.escape(name_b)}</title>
<style>
  :root {{
    --c-unchanged: #16a34a; --c-changed: #ea580c; --c-new: #2563eb;
    --c-deleted: #dc2626;
    --bg: #f7f7f8; --border: #e2e2e6;
  }}
  * {{ box-sizing: border-box; }}
  body {{ font-family: -apple-system, Segoe UI, Roboto, Arial,
sans-serif; margin: 0; background: var(--bg); color: #1a1a1a; }}
  header {{ position: sticky; top: 0; z-index: 10; background: #fff;
border-bottom: 1px solid var(--border); padding: 14px 20px; }}
  h1 {{ font-size: 16px; margin: 0 0 10px 0; font-weight: 600; }}
  .doc-names {{ font-size: 13px; color: #555; margin-bottom: 10px; }}
  .doc-names b {{ color: #111; }}
  .stats-bar {{ display: flex; gap: 10px; flex-wrap: wrap;
align-items: stretch; }}
  .stat {{ background: #fafafa; border: 1px solid var(--border);
border-radius: 8px; padding: 6px 14px; min-width: 74px; text-align:
center; }}
  .stat-num {{ font-size: 20px; font-weight: 700; }}
  .stat-label {{ font-size: 11px; color: #666; margin-top: 2px; }}
  .stat-unchanged .stat-num {{ color: var(--c-unchanged); }}
  .stat-changed .stat-num {{ color: var(--c-changed); }}
  .stat-new .stat-num {{ color: var(--c-new); }}
  .stat-deleted .stat-num {{ color: var(--c-deleted); }}
  .toolbar {{ margin-top: 10px; display: flex; gap: 14px; align-items:
center; font-size: 13px; }}
  .legend {{ display: flex; gap: 12px; font-size: 12px; color: #444;
flex-wrap: wrap; }}
  .legend span {{ display: inline-flex; align-items: center; gap: 4px;
}}
  .legend i {{ width: 10px; height: 10px; border-radius: 50%; display:
inline-block; }}
  .li-unchanged {{ background: var(--c-unchanged); }}
  .li-changed {{ background: var(--c-changed); }}
  .li-new {{ background: var(--c-new); }}
  .li-deleted {{ background: var(--c-deleted); }}
  .compare-grid {{ padding: 16px 20px 60px; }}
  .row-wrapper {{ margin-bottom: 14px; }}
  .row-wrapper.continues-group {{ margin-bottom: 0; }}
  .row-wrapper.hidden-by-filter {{ display: none; }}
  @keyframes deltaFlash {{ 0% {{ box-shadow: 0 0 0 4px rgba(234, 88,
12, 0.65); }} 100% {{ box-shadow: 0 0 0 4px rgba(234, 88, 12, 0); }}
}}

```

```

    .delta-flash {{ animation: deltaFlash 1.1s ease-out; border-radius:
8px; }}
    .grid-row {{ display: grid; grid-template-columns: 1fr 70px 1fr;
gap: 0; align-items: stretch; }}
    .cell {{ background: #fff; border: 1px solid var(--border);
border-radius: 6px; padding: 8px 12px; font-size: 13px; line-height:
1.5; overflow-wrap: break-word; }}
    .cell-slot {{ min-width: 0; }}
    .cell-left {{ border-right: none; border-radius: 6px 0 0 6px; }}
    .cell-right {{ border-left: none; border-radius: 0 6px 6px 0; }}
    .cell-empty {{ display: flex; align-items: center; justify-content:
center; color: #999; font-size: 12px; font-style: italic; background:
#fbfbfb; }}
    .page-tag {{ font-size: 18px; color: #1e293b; font-weight: 800;
letter-spacing: 0.04em; margin: 16px 0 6px 6px; text-transform:
uppercase; }}
    .pageband-odd {{ background: #f4f6fa; }}
    summary {{ cursor: pointer; font-weight: 600; }}
    .chnum {{ color: #555; font-variant-numeric: tabular-nums; }}
    .chttitle {{ color: #111; }}
    .chbody {{ margin-top: 6px; color: #333; white-space: normal; }}
    .del {{ background: #fee2e2; color: #991b1b; text-decoration:
line-through; }}
    .ins {{ background: #dcfce7; color: #14532d; }}
    .connector {{ display: flex; flex-direction: column; align-items:
center; justify-content: center; position: relative; }}
    .connector::before {{ content: ""; position: absolute; left: 0;
right: 0; top: 50%; height: 3px; transform: translateY(-50%); }}
    .conn-unchanged::before {{ background: var(--c-unchanged); }}
    .conn-changed::before {{ background: var(--c-changed); }}
    .conn-new::before {{ background: linear-gradient(to right,
transparent 50%, var(--c-new) 50%); }}
    .conn-deleted::before {{ background: linear-gradient(to right,
var(--c-deleted) 50%, transparent 50%); }}
    .connector.has-page-spine::after {{ content: ""; position: absolute;
left: 50%; top: -8px; bottom: -8px; width: 3px; background: #1e3a8a;
opacity: 0.4; transform: translateX(-50%); z-index: 0; }}
    .conn-icon {{ z-index: 1; background: #fff; border-radius: 50%;
width: 22px; height: 22px; display: flex; align-items: center;
justify-content: center; font-size: 12px; border: 2px solid; }}
    .conn-unchanged .conn-icon {{ border-color: var(--c-unchanged);
color: var(--c-unchanged); }}
    .conn-changed .conn-icon {{ border-color: var(--c-changed); color:
var(--c-changed); }}
    .conn-new .conn-icon {{ border-color: var(--c-new); color:
var(--c-new); }}
    .conn-deleted .conn-icon {{ border-color: var(--c-deleted); color:
var(--c-deleted); }}

```

```

.conn-pct {{ font-size: 10px; color: #666; margin-top: 2px; z-index:
1; }}
.conn-img-badge {{ font-size: 12px; z-index: 1; margin-top: 2px; }}
.img-gallery {{ display: flex; flex-wrap: wrap; gap: 6px;
margin-top: 8px; }}
.img-thumb {{ max-width: 160px; max-height: 120px; border: 1px solid
var(--border); border-radius: 4px; object-fit: contain; background:
#fff; }}
.img-placeholder {{ font-size: 11px; color: #92400e; background:
#fffbef; border: 1px dashed #fcd34d; border-radius: 4px; padding: 6px
10px; max-width: 220px; }}
.move-note {{ font-size: 11px; color: #6d28d9; background: #f5f3ff;
border: 1px solid #ddd6fe; border-radius: 4px; padding: 5px 9px;
margin-top: 8px; }}
.moves-summary {{ margin-top: 8px; font-size: 12px; background:
#f5f3ff; border: 1px solid #ddd6fe; border-radius: 6px; padding: 6px
12px; color: #4c1d95; }}
.moves-summary summary {{ cursor: pointer; color: #6d28d9;
font-weight: 600; }}
.moves-summary ul {{ margin: 8px 0 4px 0; padding-left: 20px; }}
.moves-summary li {{ margin-bottom: 4px; }}
.row-unchanged .cell-left, .row-unchanged .cell-right {{
border-color: #bbf7d0; }}
.row-changed .cell-left, .row-changed .cell-right {{ border-color:
#fed7aa; }}
.row-new .cell-right {{ border-color: #bfbdbfe; }}
.row-deleted .cell-left {{ border-color: #fecaca; }}
.cell.pg-grouped {{ border-left-width: 6px !important;
border-left-color: #1e3a8a !important; background: #eef2ff; }}
.cell-left.pg-grouped {{ margin-left: 6px; }}
.cell-right.pg-grouped {{ margin-right: 6px; }}
.cell.pg-mid {{ border-top: none !important; border-bottom: none
!important; border-radius: 0 !important; }}
.cell.pg-first {{ border-top: 5px solid #1e3a8a !important;
border-bottom: none !important; box-shadow: 0 -3px 8px -2px rgba(30,
58, 138, 0.35); }}
.cell.pg-last {{ border-bottom: 5px solid #1e3a8a !important;
border-top: none !important; box-shadow: 0 3px 8px -2px rgba(30, 58,
138, 0.35); }}
.cell.pg-both {{ border-top: 5px solid #1e3a8a !important;
border-bottom: 5px solid #1e3a8a !important; box-shadow: 0 0 8px -1px
rgba(30, 58, 138, 0.35); }}
.cell-left.pg-first, .cell-left.pg-both {{ border-top-left-radius:
10px; }}
.cell-left.pg-last, .cell-left.pg-mid {{ border-top-left-radius: 0;
}}
.cell-left.pg-last, .cell-left.pg-both {{ border-bottom-left-radius:
10px; }}

```

```

    .cell-left.pg-first, .cell-left.pg-mid {{ border-bottom-left-radius:
0; }}
    .cell-right.pg-first, .cell-right.pg-both {{
border-top-right-radius: 10px; }}
    .cell-right.pg-last, .cell-right.pg-mid {{ border-top-right-radius:
0; }}
    .cell-right.pg-last, .cell-right.pg-both {{
border-bottom-right-radius: 10px; }}
    .cell-right.pg-first, .cell-right.pg-mid {{
border-bottom-right-radius: 0; }}
    button.filter-btn {{ border: 1px solid var(--border); background:
#fff; border-radius: 6px; padding: 5px 12px; font-size: 12px; cursor:
pointer; }}
    button.filter-btn.active {{ background: #111; color: #fff;
border-color: #111; }}
    .review-box {{ display: flex; align-items: center; gap: 8px;
background: #fafafa; border: 1px solid var(--border); border-top:
none; border-radius: 0 0 6px 6px; padding: 6px 12px; font-size: 12px;
}}
    .review-box label {{ color: #666; white-space: nowrap; }}
    .review-select {{ font-size: 12px; padding: 3px 4px; border-radius:
4px; border: 1px solid var(--border); background: #fff; }}
    .review-comment {{ flex: 1; font-size: 12px; padding: 4px 8px;
border-radius: 4px; border: 1px solid var(--border); }}
    .review-box.rv-accepted {{ background: #f0fdf4; }}
    .review-box.rv-not_accepted {{ background: #fef2f2; }}
    .review-box.rv-refinement_customer {{ background: #eff6ff; }}
    .review-box.rv-internal_clarification {{ background: #fff7ed; }}
    .manual-link-box {{ background: #fafafa; border: 1px solid
var(--border); border-top: none; border-radius: 0 0 6px 6px; padding:
6px 12px; font-size: 12px; }}
    .ml-row {{ display: flex; align-items: center; gap: 8px; }}
    .manual-link-box label {{ color: #666; white-space: nowrap; }}
    .manual-link-input {{ flex: 1; font-size: 12px; padding: 4px 8px;
border-radius: 4px; border: 1px solid var(--border); }}
    .manual-link-box.ml-valid {{ background: #eff6ff; }}
    .manual-link-box.ml-valid .manual-link-input {{ border-color:
#2563eb; color: #1d4ed8; }}
    .manual-link-box.ml-invalid .manual-link-input {{ border-color:
#dc2626; color: #b91c1c; }}
    .link-diff {{ margin-top: 8px; padding: 8px 10px; background: #fff;
border: 1px solid #bfdbfe; border-radius: 4px; font-size: 12px;
line-height: 1.5; display: none; }}
    .link-diff.ld-visible {{ display: block; }}
    .link-diff.ld-label {{ font-size: 10px; text-transform: uppercase;
letter-spacing: 0.03em; color: #2563eb; font-weight: 700;
margin-bottom: 4px; }}
    .ml-btn {{ border: 1px solid var(--border); background: #fff;

```

```

border-radius: 4px; padding: 3px 8px; font-size: 12px; cursor:
pointer; color: #444; }}
.ml-btn:hover {{ background: #f1f5f9; }}
.version-line {{ font-size: 11px; color: #888; }}
.diagnostics-box {{ margin-top: 8px; font-size: 12px; background:
#f8fafc; border: 1px solid var(--border); border-radius: 6px; padding:
6px 12px; }}
.diagnostics-box summary {{ cursor: pointer; color: #475569;
font-weight: 600; }}
.diagnostics-box ul {{ margin: 8px 0 4px 0; padding-left: 20px;
color: #334155; }}
.diagnostics-box li {{ margin-bottom: 4px; }}
#review-import-warning {{ display: none; background: #fffbeb;
border: 1px solid #fcd34d; color: #92400e; padding: 8px 14px;
border-radius: 6px; font-size: 12px; margin-top: 8px; white-space:
pre-line; }}
</style>
</head>
<body>
<header>
  <h1>Kapitelvergleich (Fixpunkt: Kapitelnummer)</h1>
  <div class="doc-names">Dokument A: <b>{html.escape(name_a)}</b> |
Dokument B: <b>{html.escape(name_b)}</b></div>
  <div class="doc-names"><img alt="info icon" data-bbox="388 484 408 504" style="vertical-align: middle;"/> {html.escape(linebreak_note)}</div>
  {f'<div class="doc-names">{html.escape(page_note)}</div>' if
page_note else ''}
  {f'<div class="doc-names">{html.escape(toc_note)}</div>' if toc_note
else ''}
  {diagnostics_html}
  {moves_summary_html}
  <div class="version-line">Tool-Version {SCRIPT_VERSION} .
Review-Schema {REVIEW_SCHEMA_VERSION}</div>
  <div class="stats-bar">{stats_html}</div>
  <div class="toolbar">
    <button class="filter-btn active" id="btn-all"
onclick="setFilter('all')">Alle</button>
    <button class="filter-btn" id="btn-diff"
onclick="setFilter('diff')">Nur Unterschiede</button>
    <button class="filter-btn" id="btn-unreviewed"
onclick="setFilter('unreviewed')">Nur unbewertet</button>
    <button class="filter-btn" id="btn-expand"
onclick="toggleExpandAll()">Alle auf-/zuklappen</button>
    <button class="filter-btn" onclick="jumpToDelta(-1)" title="Zum
vorherigen Unterschied springen">⏮ Vorheriges Delta</button>
    <button class="filter-btn" onclick="jumpToDelta(1)" title="Zum
naechsten Unterschied springen">⏭ Nächstes Delta</button>
    <button class="filter-btn" onclick="exportReviews()">📄 Review
exportieren (JSON)</button>

```

```

        <button class="filter-btn"
onclick="document.getElementById('review-file-input').click()">
Review importieren (JSON)</button>
        <input type="file" id="review-file-input"
accept=".json,application/json" style="display:none"
onchange="importReviewsFile(event)">
        <span class="legend">
            <span><i class="li-unchanged"></i> unverändert</span>
            <span><i class="li-changed"></i> geändert</span>
            <span><i class="li-new"></i> neu</span>
            <span><i class="li-deleted"></i> gelöscht</span>
            <span>&img alt="Grafik-Symbol" data-bbox="248 278 268 298"/> Grafik geändert</span>
        </span>
    </div>
    <div id="review-import-warning"></div>
</header>
<div class="compare-grid" id="grid">
    {''.join(row_html)}
</div>
<datalist id="chapter-datalist">
    {''.join(f'<option value="{html.escape(r["number"], quote=True)}">'
for r in rows)}
</datalist>
<script>
    const DOC_META = {doc_meta_json};
    const SCRIPT_VERSION = {script_version_json};
    const REVIEW_SCHEMA_VERSION = {review_schema_json};
    const REVIEW_STATUS_VALUES = ['accepted', 'not_accepted',
'refinement_customer', 'internal_clarification'];
    const CHAPTER_KEYS = {chapter_keys_json};
    const SAFE_ID_TO_KEY = {safe_id_to_key_json};
    const ROW_TEXTS = {row_texts_json};
    const KEY_TO_SAFE_ID =
Object.fromEntries(Object.entries(SAFE_ID_TO_KEY).map(function(e) {{
return [e[1], e[0]]; }}));
    const MANUAL_LINKS = {{{}}};
    const LINK_DIFF_MAX_TOKENS = 10000;

    const REVIEW_BOXES = {{{}}};
    document.querySelectorAll('.review-box').forEach(function(box) {{
        REVIEW_BOXES[box.getAttribute('data-review-key')] = box;
    }});

    let filterMode = 'all';
    let expanded = false;
    let deltaIndex = -1;

    function setFilter(mode) {{

```

```

filterMode = mode;
deltaIndex = -1;
['all', 'diff', 'unreviewed'].forEach(function(m) {{
    document.getElementById('btn-' + m).classList.toggle('active', m
=== mode);
}});
document.querySelectorAll('.row-wrapper').forEach(function(row) {{
    const status = row.getAttribute('data-status');
    let show = true;
    if (mode === 'diff') {{
        show = status !== 'unchanged';
    }} else if (mode === 'unreviewed') {{
        const select = row.querySelector('.review-select');
        show = !select || !select.value;
    }}
    row.classList.toggle('hidden-by-filter', !show);
}});
}}

function toggleExpandAll() {{
    expanded = !expanded;
    document.querySelectorAll('details.cell').forEach(function(d) {{
d.open = expanded; }});
}}

function getDeltaRows() {{
    return
Array.from(document.querySelectorAll('.row-wrapper')).filter(function(
row) {{
    return row.getAttribute('data-status') !== 'unchanged' &&
!row.classList.contains('hidden-by-filter');
}});
}}

function jumpToDelta(direction) {{
    const rows = getDeltaRows();
    if (!rows.length) {{ return; }}
    deltaIndex += direction;
    if (deltaIndex >= rows.length) {{ deltaIndex = 0; }}
    if (deltaIndex < 0) {{ deltaIndex = rows.length - 1; }}
    const row = rows[deltaIndex];
    row.scrollIntoView({{behavior: 'smooth', block: 'center'}});
    row.classList.remove('delta-flash');
    void row.offsetWidth;
    row.classList.add('delta-flash');
}}

function onReviewChange(id) {{

```

```

    const select = document.getElementById('rs-' + id);
    const box = select.closest('.review-box');
    REVIEW_STATUS_VALUES.forEach(function(v) {{
box.classList.remove('rv-' + v); }});
    if (select.value) {{ box.classList.add('rv-' + select.value); }}
    if (filterMode === 'unreviewed') {{ setFilter('unreviewed'); }}
}}

function tokenizeWords(text) {{
    return (text || '').match(/\s+|\S+/g) || [];
}}

function escHtml(s) {{
    const d = document.createElement('div');
    d.textContent = s;
    return d.innerHTML;
}}

function myersDiffOps(a, b) {{
    const n = a.length, m = b.length;
    if (n === 0 && m === 0) {{ return []; }}
    const max = n + m;
    const offset = max;
    const size = 2 * max + 1;
    let v = new Int32Array(size);
    const trace = [];
    outer:
    for (let d = 0; d <= max; d++) {{
        trace.push(v.slice());
        for (let k = -d; k <= d; k += 2) {{
            let x;
            if (k === -d || (k !== d && v[k - 1 + offset] < v[k + 1 +
offset])) {{
                x = v[k + 1 + offset];
            }} else {{
                x = v[k - 1 + offset] + 1;
            }}
            let y = x - k;
            while (x < n && y < m && a[x] === b[y]) {{ x++; y++; }}
            v[k + offset] = x;
            if (x >= n && y >= m) {{ break outer; }}
        }}
    }}
    let x = n, y = m;
    const ops = [];
    for (let d = trace.length - 1; d >= 0; d--) {{
        const vPrev = trace[d];
        const k = x - y;

```

```

    let prevK;
    if (k === -d || (k !== d && vPrev[k - 1 + offset] < vPrev[k + 1
+ offset])) {{
        prevK = k + 1;
    }} else {{
        prevK = k - 1;
    }}
    const prevX = vPrev[prevK + offset];
    const prevY = prevX - prevK;
    while (x > prevX && y > prevY) {{
        ops.push({{op: 'equal', tok: a[x - 1]}});
        x--; y--;
    }}
    if (d > 0) {{
        if (x === prevX) {{
            ops.push({{op: 'ins', tok: b[y - 1]}});
            y--;
        }} else {{
            ops.push({{op: 'del', tok: a[x - 1]}});
            x--;
        }}
    }}
    ops.reverse();
    return ops;
}}
```

```

function wordDiffHtml(textA, textB) {{
    const a = tokenizeWords(textA);
    const b = tokenizeWords(textB);
    if (a.length + b.length > LINK_DIFF_MAX_TOKENS) {{
        return [escHtml(textA), escHtml(textB), false];
    }}
    const ops = myersDiffOps(a, b);
    const left = [], right = [];
    for (const o of ops) {{
        if (o.op === 'equal') {{
            left.push(escHtml(o.tok)); right.push(escHtml(o.tok));
        }} else if (o.op === 'del') {{
            left.push('<span class="del">' + escHtml(o.tok) + '</span>');
        }} else {{
            right.push('<span class="ins">' + escHtml(o.tok) + '</span>');
        }}
    }}
    return [left.join(''), right.join(''), true];
}}
```

```

function updateLinkDiff(id) {{
```

```

const diffBox = document.getElementById('ld-' + id);
if (!diffBox) {{ return; }}
const ownKey = SAFE_ID_TO_KEY[id];
const targetKey = MANUAL_LINKS[id];
if (!targetKey || !ROW_TEXTS[ownKey] || !ROW_TEXTS[targetKey]) {{
    diffBox.classList.remove('ld-visible');
    diffBox.innerHTML = '';
    return;
}}
const ownText = ROW_TEXTS[ownKey].a || ROW_TEXTS[ownKey].b || '';
const targetText = ROW_TEXTS[targetKey].b ||
ROW_TEXTS[targetKey].a || '';
const [leftHtml, rightHtml, wasCompared] = wordDiffHtml(ownText,
targetText);
const targetNumber = CHAPTER_KEYS[targetKey] || targetKey;
diffBox.innerHTML =
    '<div class="ld-label">' + (wasCompared ? 'Unterschied' :
'Vergleich (Text zu lang)') +
    ' zu Kapitel ' + escHtml(targetNumber) + ':</div>' +
    '<div>' + leftHtml + '</div><div style="margin-top:4px;">' +
rightHtml + '</div>';
diffBox.classList.add('ld-visible');
}}

function onManualLinkChange(id) {{
    const input = document.getElementById('ml-' + id);
    const box = input.closest('.manual-link-box');
    const val = input.value.trim();
    const ownKey = SAFE_ID_TO_KEY[id];
    box.classList.remove('ml-valid', 'ml-invalid');
    if (!val) {{
        delete MANUAL_LINKS[id];
        updateLinkDiff(id);
        return;
    }}
    const foundKey = Object.keys(CHAPTER_KEYS).find(function(k) {{
        return CHAPTER_KEYS[k] === val && k !== ownKey;
    }});
    if (foundKey) {{
        MANUAL_LINKS[id] = foundKey;
        box.classList.add('ml-valid');
    }} else {{
        delete MANUAL_LINKS[id];
        box.classList.add('ml-invalid');
    }}
    updateLinkDiff(id);
}}

```

```

function clearManualLink(id) {{
  const input = document.getElementById('ml-' + id);
  input.value = '';
  onManualLinkChange(id);
}}

function jumpToManualLink(id) {{
  const targetKey = MANUAL_LINKS[id];
  if (!targetKey) {{ return; }}
  const box = REVIEW_BOXES[targetKey];
  const row = box ? box.closest('.row-wrapper') : null;
  if (!row) {{ return; }}
  row.scrollIntoView({{behavior: 'smooth', block: 'center'}});
  row.classList.remove('delta-flash');
  void row.offsetWidth;
  row.classList.add('delta-flash');
}}

function collectReviews() {{
  const result = {};
  Object.keys(REVIEW_BOXES).forEach(function(key) {{
    const box = REVIEW_BOXES[key];
    const status = box.querySelector('.review-select').value;
    const comment =
box.querySelector('.review-comment').value.trim();
    if (status || comment) {{
      result[key] = {{status: status, comment: comment}};
    }}
  }});
  return result;
}}

function collectManualLinks() {{
  const result = {};
  Object.keys(MANUAL_LINKS).forEach(function(safeId) {{
    const ownKey = SAFE_ID_TO_KEY[safeId];
    const targetKey = MANUAL_LINKS[safeId];
    if (ownKey && targetKey) {{ result[ownKey] = targetKey; }}
  }});
  return result;
}}

function sanitizeFilename(name) {{
  return (name || 'doc').replace(/^[a-zA-Z0-9_.-]+/g, '_').slice(0,
60);
}}

function exportReviews() {{

```

```

const data = {{
  schema_version: REVIEW_SCHEMA_VERSION,
  generated_by: 'docx_chapter_compare.py v' + SCRIPT_VERSION,
  doc_a: DOC_META.a,
  doc_b: DOC_META.b,
  reviews: collectReviews(),
  manual_moves: collectManualLinks()
}};
const blob = new Blob([JSON.stringify(data, null, 2)], {{type:
'application/json'}});
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = 'review_' + sanitizeFilename(DOC_META.a.name) +
'_vs_' + sanitizeFilename(DOC_META.b.name) + '.json';
document.body.appendChild(a);
a.click();
document.body.removeChild(a);
URL.revokeObjectURL(url);
}}

function importReviewsFile(evt) {{
  const file = evt.target.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = function(e) {{
    let data;
    try {{ data = JSON.parse(e.target.result); }}
    catch (err) {{ alert('Ungültige JSON-Datei: ' + err.message);
return; }}
    applyReviews(data);
  }};
  reader.readAsText(file);
  evt.target.value = '';
}}

function applyReviews(data) {{
  data = data || {{}};
  const reviews = data.reviews || {{}};
  let matched = 0, unmatched = 0;
  Object.keys(reviews).forEach(function(key) {{
    const entry = reviews[key] || {{}};
    const box = REVIEW_BOXES[key];
    if (!box) {{ unmatched++; return; }}
    const select = box.querySelector('.review-select');
    const comment = box.querySelector('.review-comment');
    select.value = entry.status || '';
    comment.value = entry.comment || '';
  }}

```

```

        onReviewChange(select.id.replace('rs-', ''));
        matched++;
    }));

const manualMoves = data.manual_moves || {};
let linkMatched = 0, linkUnmatched = 0;
Object.keys(manualMoves).forEach(function(ownKey) {
    const targetKey = manualMoves[ownKey];
    const safeId = KEY_TO_SAFE_ID[ownKey];
    const input = safeId ? document.getElementById('ml-' + safeId) :
null;
    const targetNumber = CHAPTER_KEYS[targetKey];
    if (!input || !targetNumber) {{ linkUnmatched++; return; }}
    input.value = targetNumber;
    onManualLinkChange(safeId);
    linkMatched++;
}});

const warnBox = document.getElementById('review-import-warning');
const da = data.doc_a || {}, db = data.doc_b || {};
let warnings = [];
if (da.name && DOC_META.a.name && da.name !== DOC_META.a.name) {{
    warnings.push('Dokument A: JSON nennt "' + da.name + '"', Report
vergleicht '"' + DOC_META.a.name + '"');
}}
if (da.modified && DOC_META.a.modified && da.modified !==
DOC_META.a.modified) {{
    warnings.push('Dokument A wurde seit dem Review-Export geändert
(Zeitstempel weicht ab)');
}}
if (db.name && DOC_META.b.name && db.name !== DOC_META.b.name) {{
    warnings.push('Dokument B: JSON nennt "' + db.name + '"', Report
vergleicht '"' + DOC_META.b.name + '"');
}}
if (db.modified && DOC_META.b.modified && db.modified !==
DOC_META.b.modified) {{
    warnings.push('Dokument B wurde seit dem Review-Export geändert
(Zeitstempel weicht ab)');
}}
if (unmatched > 0) {{
    warnings.push(unmatched + ' Kapitel aus der JSON-Datei wurden im
aktuellen Report nicht gefunden und übersprungen');
}}
if (linkUnmatched > 0) {{
    warnings.push(linkUnmatched + ' manuelle Verknüpfung(en) konnten
nicht wiederhergestellt werden');
}}
if (warnings.length) {{

```

```

        warnBox.textContent = '⚠ ' + warnings.join('\n⚠ ');
        warnBox.style.display = 'block';
    }} else {{
        warnBox.style.display = 'none';
    }}
    alert('Review importiert: ' + matched + ' Kapitel, ' + linkMatched
+ ' Verknüpfung(en).');
    }}
</script>
</body>
</html>
"""

```

```

def _run_with_timeout(func, timeout, *args, **kwargs):
    result_box = {}

    def runner():
        start = time.perf_counter()
        try:
            result_box["result"] = func(*args, **kwargs)
            result_box["status"] = "ok"
        except Exception as exc:
            result_box["status"] = "error"
            result_box["error"] = f"{type(exc).__name__}: {exc}"
        result_box["duration"] = time.perf_counter() - start

    start = time.perf_counter()
    t = threading.Thread(target=runner, daemon=True)
    t.start()
    t.join(timeout)
    if t.is_alive():
        return "timeout", None, time.perf_counter() - start, f"Kein
Ergebnis nach {timeout}s"
    return (
        result_box.get("status", "error"),
        result_box.get("result"),
        result_box.get("duration", time.perf_counter() - start),
        result_box.get("error"),
    )

def _text_len_stats(chapters):
    lengths = [len(ch.get("text", "")) for ch in chapters]
    if not lengths:
        return {"count": 0}
    return {
        "count": len(lengths),
        "min_chars": min(lengths),
        "max_chars": max(lengths),
    }

```

```

        "avg_chars": round(sum(lengths) / len(lengths), 1),
        "total_chars": sum(lengths),
    }

def _source_breakdown(chapters):
    counts = {}
    for ch in chapters:
        src = ch.get("_source", "unknown")
        counts[src] = counts.get(src, 0) + 1
    return counts

def generate_diagnostic_report(path_a, path_b, output_path,
                              step_timeout=90,
                              ignore_linebreaks=True,
                              run_pages=False, run_moves=True):
    report = {
        "report_version": "1.0",
        "generated_at": datetime.now().isoformat(timespec="seconds"),
        "script_version": SCRIPT_VERSION,
        "environment": {
            "python_version": sys.version.split()[0],
            "platform": sys.platform,
            "pywin32_available": find_word_com(),
            "pdfplumber_available": _pdfplumber_available(),
            "soffice_path": find_soffice(),
        },
        "settings": {
            "ignore_linebreaks": ignore_linebreaks,
            "run_pages": run_pages,
            "run_moves": run_moves,
            "step_timeout_s": step_timeout,
        },
        "steps": {},
    }

def doc_basic_info(path):
    p = Path(path)
    try:
        size = p.stat().st_size
    except OSError:
        size = None
    info = {"size_bytes": size}
    try:
        d = Document(path)
        info["paragraph_count"] = len(d.paragraphs)
        style_counts = {}
        for para in d.paragraphs:
            sid = para.style.style_id if para.style else "?"

```

```

        style_counts[sid] = style_counts.get(sid, 0) + 1
        info["style_counts"] = style_counts
    except Exception as exc:
        info["error"] = f"{type(exc).__name__}: {exc}"
    return info

    chapters_a = chapters_b = None
    for label, path, key in (("doc_a", path_a, "chapters_a"),
                            ("doc_b", path_b, "chapters_b")):
        basic = doc_basic_info(path)
        status, result, duration, error =
_run_with_timeout(extract_chapters, step_timeout, path)
        step_report = {"basic_info": basic, "status": status,
"duration_s": round(duration, 2)}
        if status == "ok":
            step_report["chapter_count"] = len(result)
            step_report["text_len_stats"] = _text_len_stats(result)
            step_report["number_source_breakdown"] =
_source_breakdown(result)
            step_report["image_count_total"] =
sum(len(ch.get("images", [])) for ch in result)
            if key == "chapters_a":
                chapters_a = result
            else:
                chapters_b = result
        else:
            step_report["error"] = error
        report["steps"][label] = step_report

    if chapters_a is None or chapters_b is None:
        report["steps"]["aborted"] = "Kapitel-Extraktion fuer
mindestens ein Dokument fehlgeschlagen/Timeout."
        report_path = Path(output_path)
        report_path.write_text(json.dumps(report, indent=2,
ensure_ascii=False), encoding="utf-8")
        return report_path

    if run_pages:
        status, _, duration, error = _run_with_timeout(
            attach_pages, step_timeout, chapters_a, chapters_b,
path_a, path_b
        )
        report["steps"]["attach_pages"] = {"status": status,
"duration_s": round(duration, 2), "error": error}

    status, rows, duration, error = _run_with_timeout(
        build_comparison, step_timeout, chapters_a, chapters_b,
ignore_linebreaks

```

```

)
build_step = {"status": status, "duration_s": round(duration, 2)}
if status == "ok":
    stats = compute_stats(chapters_a, chapters_b, rows)
    build_step.update({
        "unchanged": stats["unchanged"], "changed":
stats["changed"],
        "new": stats["new"], "deleted": stats["deleted"],
        "fallback_candidates_a": sum(1 for ch in chapters_a if
ch["number"].startswith("_")),
        "fallback_candidates_b": sum(1 for ch in chapters_b if
ch["number"].startswith("_")),
    })
else:
    build_step["error"] = error
    report["steps"]["build_comparison"] = build_step

    if run_moves and status == "ok":
        removed_candidates = sum(1 for r in rows if r["status"] in
("changed", "deleted"))
        added_candidates = sum(1 for r in rows if r["status"] in
("changed", "new"))
        mstatus, mresult, mduration, merror =
_run_with_timeout(detect_possible_moves, step_timeout, rows)
        moves, moves_complete = (mresult if mresult is not None else
(None, None))
        report["steps"]["detect_possible_moves"] = {
            "status": mstatus, "duration_s": round(mduration, 2),
            "removed_candidate_rows": removed_candidates,
            "added_candidate_rows": added_candidates,
            "worst_case_comparisons": removed_candidates *
added_candidates,
            "moves_found": len(moves) if mstatus == "ok" and moves is
not None else None,
            "internal_time_budget_exhausted": (moves_complete is
False),
            "error": merror,
        }

    if status == "ok":
        rstatus, out_html, rduration, rerror = _run_with_timeout(
            render_html, step_timeout, rows, compute_stats(chapters_a,
chapters_b, rows),
            Path(path_a).name, Path(path_b).name,
        )
        report["steps"]["render_html"] = {
            "status": rstatus, "duration_s": round(rduration, 2),
            "output_size_bytes": len(out_html) if rstatus == "ok" else

```

```

None,
    "error": rerror,
}

    report_path = Path(output_path)
    report_path.write_text(json.dumps(report, indent=2,
ensure_ascii=False), encoding="utf-8")
    return report_path

def _pdfplumber_available():
    try:
        import pdfplumber # noqa: F401
        return True
    except ImportError:
        return False

#
-----
----
# CLI
#
-----
----

def diagnose_page_detection(soffice_explicit=None):
    lines = []

    if sys.platform != "win32":
        lines.append(f"MS Word/COM: nicht Windows (Plattform:
{sys.platform}), daher nicht verfuegbar.")
    else:
        try:
            import win32com.client
            import pythoncom
        except ImportError:
            lines.append("MS Word/COM: pywin32 ist NICHT installiert.
Installieren mit: pip install pywin32")
        else:
            word = None
            try:
                pythoncom.CoInitialize()
                word = win32com.client.DispatchEx("Word.Application")
                lines.append("MS Word/COM: Verbindung zu Word
erfolgreich aufgebaut - sollte also funktionieren.")
            except Exception as exc:
                lines.append(f"MS Word/COM: pywin32 ist installiert,
aber die Verbindung zu Word schlug fehl: {exc!r}.")
            finally:

```

```

        try:
            if word is not None and word.Documents.Count == 0:
                word.Quit()
        except Exception:
            pass
        try:
            pythoncom.CoUninitialize()
        except Exception:
            pass

soffice_path = find_soffice(soffice_explicit)
if soffice_path:
    lines.append(f"LibreOffice: soffice gefunden unter
'{{soffice_path}}'.")
    try:
        import pdfplumber # noqa: F401
        pdfplumber_ok = True
        lines.append("pdfplumber: ist installiert - PDF-Seiten
koennen gelesen werden.")
    except ImportError:
        pdfplumber_ok = False
        lines.append("pdfplumber: ist NICHT installiert!
Installieren mit: pip install pdfplumber")

    if pdfplumber_ok:
        try:
            with tempfile.TemporaryDirectory() as tmp:
                test_docx = Path(tmp) / "diagnose_test.docx"
                Document_ = Document()
                Document_.add_paragraph("1\tTestabsatz fuer
Diagnose")

                Document_.save(test_docx)
                result = render_page_texts(test_docx,
soffice_path, timeout=60)
                if result:
                    lines.append("End-zu-End-Test: Testkonvertierung
erfolgreich.")
                else:
                    lines.append("End-zu-End-Test: Testkonvertierung
ist FEHLGESCHLAGEN.")
        except Exception as exc:
            lines.append(f"End-zu-End-Test: Fehler beim Testen:
{{exc!r}}")
        else:
            lines.append("LibreOffice: soffice wurde NICHT gefunden.")
    return lines

```

```

def main():
    print(f"docx_chapter_compare.py Version {SCRIPT_VERSION}"
    (f"({Path(__file__).resolve()})")
    parser = argparse.ArgumentParser(description="Vergleicht zwei
Word-Dokumente anhand von Kapitelnummern.")
    parser.add_argument("doc_a", nargs="?", help="Pfad zum ersten
(alten) .docx")
    parser.add_argument("doc_b", nargs="?", help="Pfad zum zweiten
(neuen) .docx")
    parser.add_argument("-o", "--output", default="vergleich.html",
help="Pfad der HTML-Ausgabedatei")
    parser.add_argument("--keep-linebreaks", action="store_true",
help="Zeilen-/Absatzumbrueche NICHT ignorieren.")
    parser.add_argument("--no-pages", action="store_true",
help="Seiten-Gruppierung abschalten.")
    parser.add_argument("--no-moves", action="store_true",
help="Verschiebungs-Erkennung abschalten.")
    parser.add_argument("--soffice-path", default=None,
help="Expliziter Pfad zu soffice.exe.")
    parser.add_argument("--diagnose-pages", action="store_true",
help="Nur Diagnose fuer Seiten-Gruppierung.")
    parser.add_argument("--docx", nargs="?", const="", default=None,
metavar="PFAD", help="Word-Report erzeugen.")
    parser.add_argument("--review-json", default=None, metavar="PFAD",
help="Review JSON-Datei einlesen.")
    parser.add_argument("--diagnostic-report", nargs="?", const="",
default=None, metavar="PFAD", help="Diagnose-Report erzeugen.")
    parser.add_argument("--diagnostic-step-timeout", type=int,
default=90, metavar="SEKUNDEN", help="Timeout je Schritt.")
    args = parser.parse_args()

    if args.diagnose_pages:
        print("Diagnose Seiten-Gruppierung:")
        for line in diagnose_page_detection(args soffice_path):
            print(f" - {line}")
        sys.exit(0)

    if not args.doc_a or not args.doc_b:
        parser.error("doc_a und doc_b sind erforderlich (ausser bei
--diagnose-pages).")

    if args.diagnostic_report is not None:
        path_a, path_b = Path(args.doc_a), Path(args.doc_b)
        for p in (path_a, path_b):
            if not p.exists():
                print(f"Datei nicht gefunden: {p}", file=sys.stderr)
                sys.exit(1)
        out_path = Path(args.diagnostic_report or

```

```

"diagnose_report.json")
    report_path = generate_diagnostic_report(
        path_a, path_b, out_path,
        step_timeout=args.diagnostic_step_timeout,
        ignore_linebreaks=not args.keep_linebreaks,
        run_pages=not args.no_pages, run_moves=not args.no_moves,
    )
    print(f"Diagnose-Report geschrieben: {report_path.resolve()}")
    sys.exit(0)

ignore_linebreaks = not args.keep_linebreaks
path_a, path_b = Path(args.doc_a), Path(args.doc_b)
for p in (path_a, path_b):
    if not p.exists():
        print(f"Datei nicht gefunden: {p}", file=sys.stderr)
        sys.exit(1)

chapters_a = extract_chapters(path_a)
chapters_b = extract_chapters(path_b)

pages_method = None
diagnostics = None
toc_info = None
if not args.no_pages:
    diagnostics = diagnose_page_detection(args soffice_path)
    method, toc_info = attach_pages(chapters_a, chapters_b,
path_a, path_b, soffice_path=args soffice_path)
    pages_method = method if method is not None else "unavailable"

rows = build_comparison(chapters_a, chapters_b,
ignore_linebreaks=ignore_linebreaks)
stats = compute_stats(chapters_a, chapters_b, rows)
moves = None
moves_complete = True
if not args.no_moves:
    moves, moves_complete = detect_possible_moves(rows)

out_html = render_html(
    rows, stats, path_a.name, path_b.name,
ignore_linebreaks=ignore_linebreaks,
    meta_a=doc_metadata(path_a), meta_b=doc_metadata(path_b),
    pages_method=pages_method, diagnostics=diagnostics,
moves=moves, moves_complete=moves_complete,
    toc_info=toc_info,
)
out_path = Path(args.output)
out_path.write_text(out_html, encoding="utf-8")

```

```

print(f"Report geschrieben: {out_path.resolve()}")

if args.docx is not None:
    docx_path = Path(args.docx) if args.docx else
out_path.with_suffix(".docx")
    reviews = None
    if args.review_json:
        try:
            review_data =
json.loads(Path(args.review_json).read_text(encoding="utf-8"))
            reviews = review_data.get("reviews", {})
        except Exception as exc:
            print(f"Warnung: Review-JSON konnte nicht gelesen
werden ({exc})", file=sys.stderr)
            generate_docx_report(
                rows, stats, path_a.name, path_b.name, docx_path,
                meta_a=doc_metadata(path_a), meta_b=doc_metadata(path_b),
                reviews=reviews,
            )
            print(f"Word-Report geschrieben: {docx_path.resolve()}")

if __name__ == "__main__":
    main()

```

2. Die Datei docx_chapter_compare_gui.py (Version 2.7)

```

#!/usr/bin/env python3
"""
docx_chapter_compare_gui.py
Version 2.7 / 2026-09-21 / Grund: Update passend zu
docx_chapter_compare.py Version 3.10
"""

import json
import os
import subprocess
import sys
import threading
import tkinter as tk
import webbrowser
from pathlib import Path
from tkinter import filedialog, messagebox, ttk

GUI_VERSION = "2.7"

try:
    import docx_chapter_compare as _core

```

```

from docx_chapter_compare import (
    attach_pages,
    build_comparison,
    compute_stats,
    detect_possible_moves,
    doc_metadata,
    extract_chapters,
    generate_docx_report,
    render_html,
)
except ImportError as exc:
    print(f"Fehler beim Import von docx_chapter_compare.py: {exc}",
          file=sys.stderr)
    print("Pruefe, ob 'python-docx' installiert ist (pip install
python-docx) "
          "und ob docx_chapter_compare.py im selben Ordner liegt.",
          file=sys.stderr)
    raise

CORE_VERSION = getattr(_core, "SCRIPT_VERSION", "?")
CORE_PATH = getattr(_core, "__file__", "?")
GUI_PATH = str(Path(__file__).resolve())

print(f"docx_chapter_compare_gui.py Version {GUI_VERSION}
({GUI_PATH})")
print(f"docx_chapter_compare.py      Version {CORE_VERSION}
({CORE_PATH})")

CONFIG_PATH = Path.home() / ".docx_chapter_compare_gui.json"

def default_output_dir():
    docs = Path.home() / "Documents"
    return docs if docs.is_dir() else Path.home()

_BROWSER_EXE_NAMES = ["msedge.exe", "chrome.exe", "firefox.exe"]
_BROWSER_CANDIDATES = [
    r"%ProgramFiles(x86)%\Microsoft\Edge\Application\msedge.exe",
    r"%ProgramFiles%\Microsoft\Edge\Application\msedge.exe",
    r"%ProgramFiles%\Google\Chrome\Application\chrome.exe",
    r"%ProgramFiles(x86)%\Google\Chrome\Application\chrome.exe",
    r"%LocalAppData%\Google\Chrome\Application\chrome.exe",
    r"%ProgramFiles%\Mozilla Firefox\firefox.exe",
    r"%ProgramFiles(x86)%\Mozilla Firefox\firefox.exe",
]

def _find_browser_via_registry():
    if sys.platform != "win32":
        return None

```

```

try:
    import winreg
except ImportError:
    return None
for exe_name in _BROWSER_EXE_NAMES:
    key_path = rf"SOFTWARE\Microsoft\Windows\CurrentVersion\App
Paths\{exe_name}"
    for hive in (winreg.HKEY_LOCAL_MACHINE,
winreg.HKEY_CURRENT_USER):
        try:
            with winreg.OpenKey(hive, key_path) as key:
                path, _ = winreg.QueryValueEx(key, "")
                if path and Path(path).exists():
                    return path
        except (FileNotFoundError, OSError):
            continue
return None

def _start_local_server(source_file):
    try:
        import functools
        import http.server
        import shutil
        import tempfile
    except ImportError:
        return None, None
    try:
        temp_dir = tempfile.mkdtemp(prefix="docx_compare_view_")
        shutil.copy2(source_file, Path(temp_dir) / source_file.name)
        handler =
functools.partial(http.server.SimpleHTTPRequestHandler,
directory=temp_dir)
        httpd = http.server.HTTPServer(("127.0.0.1", 0), handler)
        port = httpd.server_address[1]
        thread = threading.Thread(target=httpd.serve_forever,
daemon=True)
        thread.start()
        return port, temp_dir
    except Exception:
        return None, None

def open_in_browser(path):
    server_port, _server_temp_dir = _start_local_server(path)
    if server_port is not None:
        url = f"http://127.0.0.1:{server_port}/{path.name}"
    else:
        url = path.resolve().as_uri()

```

```

if sys.platform == "win32":
    registry_hit = _find_browser_via_registry()
    if registry_hit:
        try:
            subprocess.Popen([registry_hit, url])
            return True, ("server" if server_port else "registry")
        except Exception:
            pass
    for template in _BROWSER_CANDIDATES:
        exe = os.path.expandvars(template)
        if Path(exe).exists():
            try:
                subprocess.Popen([exe, url])
                return True, ("server" if server_port else
"candidate_path")
            except Exception:
                continue
    try:
        webbrowser.open(url)
        return True, ("server" if server_port else "os_default")
    except Exception:
        return False, "failed"

def load_config():
    try:
        return json.loads(CONFIG_PATH.read_text(encoding="utf-8"))
    except Exception:
        return {}

def save_config(cfg):
    try:
        CONFIG_PATH.write_text(json.dumps(cfg, ensure_ascii=False,
indent=2), encoding="utf-8")
    except Exception:
        pass

class CompareApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title(f"DOCX Kapitelvergleich - GUI v{GUI_VERSION} .
Core v{CORE_VERSION}")
        self.geometry("640x500")
        self.minsize(560, 420)
        self.resizable(True, False)

        self.cfg = load_config()

```

```

        self.var_a = tk.StringVar(value=self.cfg.get("doc_a", ""))
        self.var_b = tk.StringVar(value=self.cfg.get("doc_b", ""))
        self.var_out = tk.StringVar(value=self.cfg.get("out",
str(default_output_dir() / "vergleich.html")))
        self.var_status = tk.StringVar(value="Bereit.")
        self.var_ignore_linebreaks =
tk.BooleanVar(value=self.cfg.get("ignore_linebreaks", True))
        self.var_detect_pages =
tk.BooleanVar(value=self.cfg.get("detect_pages", True))
        self.var_detect_moves =
tk.BooleanVar(value=self.cfg.get("detect_moves", True))
        self.var_export_docx =
tk.BooleanVar(value=self.cfg.get("export_docx", False))

        self._build_ui()
        self._show_splash_and_start()

def _show_splash_and_start(self):
    self.withdraw()

    splash = tk.Toplevel(self)
    splash.overridereDIRECT(True)
    splash.attributes('-topmost', True)

    width, height = 450, 300
    x = (splash.winfo_screenwidth() - width) // 2
    y = (splash.winfo_screenheight() - height) // 2
    splash.geometry(f"{width}x{height}+{x}+{y}")
    splash.configure(background="white")

    frame = tk.Frame(splash, bg="white")
    frame.pack(fill="both", expand=True)

    script_dir = Path(__file__).resolve().parent
    logo_path = script_dir / "FirmenLogo.JPG"
    logo_loaded = False

    self._splash_label = tk.Label(frame, bg="white")
    self._splash_label.place(x=0, y=0, width=width, height=height)

    version_label = tk.Label(
        frame,
        text=f"Lade Version {GUI_VERSION} (Core
{CORE_VERSION})...",
        font=("Arial", 10, "bold"),
        bg="#555555",
        fg="white",

```

```

        padx=10,
        pady=4
    )
    version_label.place(relx=0.5, rely=0.9, anchor="center")

    if logo_path.exists():
        try:
            from PIL import Image, ImageTk, ImageDraw
            import math

            img = Image.open(logo_path).convert("RGBA")
            resample_filter = getattr(Image, 'Resampling',
Image).LANCZOS
            img = img.resize((width, height), resample_filter)
            w, h = img.size

            spot_size = int(max(w, h) * 0.8)
            spot = Image.new("RGBA", (spot_size, spot_size), (255,
255, 255, 0))
            draw = ImageDraw.Draw(spot)

            cx, cy = spot_size // 2, spot_size // 2
            radius = spot_size // 2

            for r_step in range(radius, 0, -2):
                dist = r_step / radius
                alpha = int(170 * (1 - dist**2))
                draw.ellipse(
                    [(cx - r_step, cy - r_step), (cx + r_step, cy
+ r_step)],
                    fill=(255, 255, 255, alpha)
                )

            self._anim_frame = 0
            self._anim_max_frames = 100

            def update_animation():
                if not splash.wininfo_exists():
                    return

                progress = (self._anim_frame %
self._anim_max_frames) / self._anim_max_frames

                if progress < 0.65:
                    eff_progress = progress / 0.65
                    current_x = int(-spot_size + eff_progress * (w
+ spot_size))

                    base_y = (h - spot_size) / 2

```

```

        current_y = int(base_y + math.sin(eff_progress
* math.pi) * (h * 0.15))

        overlay = Image.new("RGBA", (w, h), (255, 255,
255, 0))

        overlay.paste(spot, (current_x, current_y),
spot)

        composite = Image.alpha_composite(img,
overlay)

        photo = ImageTk.PhotoImage(composite)
    else:
        photo = ImageTk.PhotoImage(img)

        self._splash_label.config(image=photo)
        self._splash_label.image = photo

        self._anim_frame += 1
        splash.after(33, update_animation)

    update_animation()
    logo_loaded = True

    except ImportError:
        self._splash_label.config(text="FirmenLogo.JPG
gefunden, aber 'Pillow' fehlt!\n'pip install Pillow'", fg="red")
    except Exception as e:
        self._splash_label.config(text=f"Fehler beim
Laden:\n{e}", fg="red")
    else:
        self._splash_label.config(text="Kein FirmenLogo.JPG im
Ordner gefunden.", fg="gray")

    if not logo_loaded:
        fallback_label = tk.Label(frame, text="Kapitelvergleich
Tool", font=("Arial", 16, "bold"), bg="white")
        fallback_label.place(relx=0.5, rely=0.4, anchor="center")

    self.after(5000, lambda: self._close_splash(splash))

def _close_splash(self, splash):
    splash.destroy()
    self.deiconify()

def _build_ui(self):
    frame = ttk.Frame(self)
    frame.pack(fill="both", expand=True)

```

```

        ttk.Label(frame, text="Kapitelvergleich zweier
Word-Dokumente", font=("", 12, "bold")).grid(
            row=0, column=0, columnspan=3, sticky="w", padx=10,
pady=(12, 4)
        )
        ttk.Label(
            frame,
            text="Kapitel-/Anforderungsnummer wird als Fixpunkt
genutzt. Ergebnis öffnet sich im Browser.",
            foreground="#555",
        ).grid(row=1, column=0, columnspan=3, sticky="w", padx=10,
pady=(0, 10))

        version_frame = ttk.Frame(frame, relief="groove",
borderwidth=1)
        version_frame.grid(row=2, column=0, columnspan=3, sticky="ew",
padx=10, pady=(0, 10))
        ttk.Label(
            version_frame,
            text=f"GUI v{GUI_VERSION} → {GUI_PATH}",
            font=("Consolas", 8), foreground="#333",
        ).pack(anchor="w", padx=6, pady=(4, 0))
        ttk.Label(
            version_frame,
            text=f"Core v{CORE_VERSION} → {CORE_PATH}",
            font=("Consolas", 8), foreground="#333",
        ).pack(anchor="w", padx=6, pady=(0, 4))

        self._file_row(frame, row=3, label="Dokument A (alt):",
var=self.var_a, command=self.pick_a)
        self._file_row(frame, row=4, label="Dokument B (neu):",
var=self.var_b, command=self.pick_b)
        self._file_row(frame, row=5, label="Report-Ausgabe:",
var=self.var_out, command=self.pick_out, save=True)

        ttk.Checkbutton(
            frame, text="Zeilenumbrüche ignorieren (nur Wortinhalt
vergleichen)",
            variable=self.var_ignore_linebreaks,
        ).grid(row=6, column=0, columnspan=3, sticky="w", padx=10,
pady=(2, 0))

        ttk.Checkbutton(
            frame, text="Seiten-Gruppierung / Buchform (MS Word oder
LibreOffice, etwas langsamer)",
            variable=self.var_detect_pages,
        ).grid(row=7, column=0, columnspan=3, sticky="w", padx=10,

```

```

pady=(2, 0))

        ttk.Checkbutton(
            frame, text="Mögliche Kapitel-Verschiebungen erkennen",
            variable=self.var_detect_moves,
        ).grid(row=8, column=0, columnspan=3, sticky="w", padx=10,
pady=(2, 0))

        ttk.Checkbutton(
            frame, text="Zusätzlich Word-Report erzeugen (kompakt, für
schnelle Weitergabe)",
            variable=self.var_export_docx,
        ).grid(row=9, column=0, columnspan=3, sticky="w", padx=10,
pady=(2, 0))

        frame.columnconfigure(1, weight=1)

        self.btn_compare = ttk.Button(frame, text="Vergleichen ►",
command=self.run_compare)
        self.btn_compare.grid(row=10, column=0, padx=10, pady=(16, 4),
sticky="w")

        self.progress = ttk.Progressbar(frame, mode="determinate",
maximum=1, value=0)
        self.progress.grid(row=10, column=1, columnspan=2, padx=10,
pady=(16, 4), sticky="ew")

        self.progress_label = ttk.Label(frame, text="",
foreground="#666", font=("", 9))
        self.progress_label.grid(row=11, column=0, columnspan=3,
sticky="w", padx=10)

        self.stats_label = ttk.Label(frame, text="", justify="left")
        self.stats_label.grid(row=12, column=0, columnspan=3,
sticky="w", padx=10, pady=(6, 0))

        status_bar = ttk.Label(self, textvariable=self.var_status,
relief="sunken", anchor="w")
        status_bar.pack(fill="x", side="bottom")

        def _file_row(self, parent, row, label, var, command, save=False):
            ttk.Label(parent, text=label).grid(row=row, column=0,
sticky="w", padx=10, pady=6)
            entry = ttk.Entry(parent, textvariable=var)
            entry.grid(row=row, column=1, sticky="ew", padx=(0, 6),
pady=6)
            ttk.Button(parent, text="Speichern unter..." if save else
"Durchsuchen...", command=command).grid(

```

```

        row=row, column=2, sticky="e", padx=(0, 10), pady=6
    )

    def pick_a(self):
        path = filedialog.askopenfilename(
            title="Dokument A (alt) auswählen",
            filetypes=[("Word-Dokumente", "*.docx"), ("Alle Dateien",
"*.*)"],
        )
        if path:
            self.var_a.set(path)

    def pick_b(self):
        path = filedialog.askopenfilename(
            title="Dokument B (neu) auswählen",
            filetypes=[("Word-Dokumente", "*.docx"), ("Alle Dateien",
"*.*)"],
        )
        if path:
            self.var_b.set(path)

    def pick_out(self):
        path = filedialog.asksaveasfilename(
            title="Report speichern unter",
            defaultextension=".html",
            filetypes=[("HTML-Datei", "*.html")],
            initialfile="vergleich.html",
        )
        if path:
            self.var_out.set(path)

    def run_compare(self):
        path_a = Path(self.var_a.get().strip())
        path_b = Path(self.var_b.get().strip())
        out_path = Path(self.var_out.get().strip() or
(default_output_dir() / "vergleich.html"))

        if out_path.suffix.lower() != ".html":
            out_path = out_path.with_name(out_path.name + ".html") if
out_path.suffix else out_path.with_suffix(".html")
            self.var_out.set(str(out_path))

        if not self.var_a.get().strip() or not
self.var_b.get().strip():
            messagebox.showwarning("Fehlende Angabe", "Bitte beide
Dokumente auswählen.")
            return
        if not path_a.exists():

```

```

        messagebox.showerror("Datei nicht gefunden", f"Dokument A
nicht gefunden:\n{path_a}")
        return
    if not path_b.exists():
        messagebox.showerror("Datei nicht gefunden", f"Dokument B
nicht gefunden:\n{path_b}")
        return

    ignore_linebreaks = self.var_ignore_linebreaks.get()
    detect_pages = self.var_detect_pages.get()
    detect_moves = self.var_detect_moves.get()
    export_docx = self.var_export_docx.get()

    steps = ["Dokument A einlesen", "Dokument B einlesen"]
    if detect_pages:
        steps += ["Preflight-Check Seiten-Gruppierung", "Seiten
ermitteln"]
    steps += ["Vergleich berechnen"]
    if detect_moves:
        steps += ["Verschiebungen erkennen"]
    steps += ["Report schreiben"]
    if export_docx:
        steps += ["Word-Report erzeugen"]

    self.btn_compare.config(state="disabled")
    self.progress.config(mode="determinate", maximum=len(steps),
value=0)
    self.progress_label.config(text=f"[0/{len(steps)}] Bereit ...")
    self.var_status.set("Vergleiche Dokumente ...")
    self.stats_label.config(text="")

    thread = threading.Thread(
        target=self._worker,
        args=(path_a, path_b, out_path, ignore_linebreaks,
detect_pages, detect_moves, export_docx, len(steps)),
        daemon=True,
    )
    thread.start()

    def _set_progress(self, i, total, label):
        self.progress["value"] = i
        self.progress_label.config(text=f"[{i}/{total}] {label} ...")

    def _worker(self, path_a, path_b, out_path, ignore_linebreaks,
detect_pages, detect_moves, export_docx, total_steps):
        step_counter = [0]

        def advance(label):

```

```

        step_counter[0] += 1
        i = step_counter[0]
        self.after(0, lambda: self._set_progress(i, total_steps,
label))

    try:
        chapters_a = extract_chapters(path_a)
        advance("Dokument A einlesen")
        chapters_b = extract_chapters(path_b)
        advance("Dokument B einlesen")

        if not chapters_a or not chapters_b:
            self.after(0, lambda: messagebox.showwarning(
                "Keine Kapitel erkannt",
                "In mindestens einem Dokument wurden keine
Kapitel/Anforderungen erkannt.\n"
                "Ergebnis kann unvollständig sein.",
            ))

        pages_method = None
        diagnostics = None
        toc_info = None
        if detect_pages:
            diagnostics = _core.diagnose_page_detection()
            advance("Preflight-Check Seiten-Gruppierung")
            method, toc_info = attach_pages(chapters_a,
chapters_b, path_a, path_b)
            pages_method = method if method is not None else
"unavailable"
            advance("Seiten ermitteln")

            rows = build_comparison(chapters_a, chapters_b,
ignore_linebreaks=ignore_linebreaks)
            advance("Vergleich berechnen")

            moves = None
            moves_complete = True
            if detect_moves:
                moves, moves_complete = detect_possible_moves(rows)
                advance("Verschiebungen erkennen")

            stats = compute_stats(chapters_a, chapters_b, rows)
            out_html = render_html(
                rows, stats, path_a.name, path_b.name,
ignore_linebreaks=ignore_linebreaks,
                meta_a=doc_metadata(path_a),
meta_b=doc_metadata(path_b),
                pages_method=pages_method, diagnostics=diagnostics,

```

```

moves=moves, moves_complete=moves_complete,
        toc_info=toc_info,
    )
    out_path.parent.mkdir(parents=True, exist_ok=True)
    out_path.write_text(out_html, encoding="utf-8")
    advance("Report schreiben")

    docx_path = None
    if export_docx:
        docx_path = out_path.with_suffix(".docx")
        generate_docx_report(
            rows, stats, path_a.name, path_b.name, docx_path,
            meta_a=doc_metadata(path_a),
            meta_b=doc_metadata(path_b),
        )
        advance("Word-Report erzeugen")

    save_config({
        "doc_a": str(path_a), "doc_b": str(path_b), "out":
str(out_path),
        "ignore_linebreaks": ignore_linebreaks,
        "detect_pages": detect_pages,
        "detect_moves": detect_moves, "export_docx":
export_docx,
    })

    self.after(0, lambda: self._on_success(out_path, stats,
docx_path,
                                                    len(moves) if
moves else 0, moves_complete))
    except Exception as exc:
        self.after(0, lambda: self._on_error(exc))

    def _on_success(self, out_path, stats, docx_path=None,
move_count=0, moves_complete=True):
        self.btn_compare.config(state="normal")
        status_text = f"Report erstellt: {out_path}"
        if docx_path:
            status_text += f" | Word-Report: {docx_path}"
        self.var_status.set(status_text)
        move_note = ""
        if move_count:
            move_note = f"  {move_count} mögliche Verschiebung(en)
erkannt."
        if not moves_complete:
            move_note += " ⚠ Verschiebungs-Erkennung wegen Zeitbudget
unvollständig."
        self.progress_label.config(text="Fertig." + move_note)

```

```

        self.stats_label.config(
            text=(
                f"Kapitel A: {stats['total_a']}    Kapitel B:
{stats['total_b']}    "
                f"Unverändert: {stats['unchanged']}    Geändert:
{stats['changed']}    "
                f"Neu: {stats['new']}    Gelöscht: {stats['deleted']}"
            )
        )
        opened, method = open_in_browser(out_path)
        if not opened:
            messagebox.showinfo(
                "Bericht bereit",
                f"Der Bericht wurde erstellt, konnte aber nicht
automatisch geöffnet werden:\n\n"
                f"{out_path}\n\nBitte die Datei manuell doppelklicken
oder in einen Browser ziehen.",
            )
        elif method == "os_default":
            messagebox.showwarning(
                "Bericht geöffnet über Windows-Standard",
                "Browser nicht erkannt. Falls sich der Bericht nicht
korrekt öffnet, ziehe ihn manuell in Microsoft Edge oder Chrome:\n\n"
                f"{out_path}",
            )

    def _on_error(self, exc):
        self.btn_compare.config(state="normal")
        self.var_status.set("Fehler beim Vergleich.")
        self.progress_label.config(text="Abgebrochen.")
        messagebox.showerror("Fehler", f"Beim Vergleich ist ein Fehler
aufgetreten:\n\n{exc}")

def main():
    app = CompareApp()
    app.mainloop()

if __name__ == "__main__":
    main()

```